DevOps Question Bank Terraform & Infrastructure as Code

48 real interview questions - de-duplicated, topic-wise, Basic / Advanced / Scenario-Based
Every answer: plain-English idea · technical answer · example · what to say in the interview

By Priyanka Gupta — 25 September 2026

How to use this question bank

Each question has four parts:

In simple words (green box) - a plain-English analogy. Read this first; if you understand it, the technical part becomes easy to remember.

Detailed answer + Example - the real technical answer, with commands / YAML / code you can show.

What to say to the interviewer (blue box) - 2-3 lines to say out loud. Practise these aloud.

Questions that were asked more than once across the lists were merged into one (the other wordings are shown in grey as 'Also asked as'). Language-specific (Python/PHP) and non-DevOps questions were removed. Where a pasted 'model answer' was wrong, the corrected version is given.

Suggested order: all Basic questions first, then Scenario-Based (most interviews are troubleshooting-heavy), then Advanced. Use the PDF bookmarks panel to jump to any question.

Topic	Basic	Advanced	Scenario	Total
Terraform & Infrastructure as Code	15	20	13	48

Questions in this topic

Basic **5**

Q1. What is Infrastructure as Code, and how do you manage it using Terraform or CloudFormation? . . . 5

Q2. What is the Terraform state file and why is it important? . . . 6

Q3. What are the two main types of variables in Terraform? How are locals and outputs different? . . . 7

Q4. What is the difference between count and for_each, and what is a for expression? When do you use each? . . . 8

Q5. What is the difference between variables.tf and .tfvars files? What is the variable precedence order? 9

Q6. In which Terraform file or block can you identify the provider version (for example the GCP google provider)? . . . 10

Q7. What files are in a typical Terraform folder or module, and what is each file for? . . . 11

Q8. How does Terraform prevent duplicate resource creation? . . . 12

Q9. Write Terraform code to launch an EC2 instance. . . . 12

Q10. What is drift in Terraform? . . . 14

Q11. What is the use of the terraform refresh command? . . . 14

Q12. What is a null_resource in Terraform? . . . 15

Q13. What is the GCP (and Azure) equivalent of AWS CloudFormation? . . . 15

Q14. What are some non-cloud-native alternatives to Terraform? . . . 16

Q15. What is immutable infrastructure, and how do Packer and Terraform help? . . . 17

Advanced **18**

Q16. How do you set up remote state and state locking, especially when multiple engineers modify infrastructure at the same time? . . . 18

Q17. How do you manage existing (unmanaged) AWS resources in Terraform? What is the difference between terraform import and an import block? . . . 19

Q18. How do you dynamically retrieve VPC details from AWS to create an EC2 instance using IaC? Write the code. . . . 20

Q19. If you want to replace (recreate) a single resource in existing infrastructure, how would you do it? . 21

Q20. How do you control precisely which resources are destroyed and which are kept? . . . 21

Q21. Besides depends_on, how do you control the order in which resources are deployed? Can you add a time delay? . . . 23

Q22. How do you design Terraform modules and keep code DRY? . . . 24

Q23. How do you manage separate environments (Dev, Test, Prod) in Terraform? Workspaces or separate directories? . . . 25

Q24. How would you create resources across two different AWS accounts using Terraform? . . . 26

Q25. How do you manage sensitive values like secrets or API keys in Terraform without hardcoding them? 27

Q26. How do you upgrade a GKE node pool version using Terraform (and manually)? . . . 28

Q27. How do you create an EKS cluster using Terraform, and where do Kubernetes core components come into play? . . . 29

Q28. You wrote Terraform for an EKS cluster plus a Kubernetes service account. The IAM role must exist before the cluster, and IRSA needs the OIDC provider, which only exists after the cluster. How do you solve the ordering? . . . 30

Q29. Write Terraform code to deploy an EKS cluster, a Helm chart and the Datadog agent, and use the agent for monitoring. . . . 33

Q30. How do you detect drift in Terraform and troubleshoot it? . . . 35

Q31. How do you integrate Terraform with CI/CD and safely deploy infrastructure changes to production? 36

Q32. What are RPO and RTO, and how does Terraform / IaC help you meet them in disaster recovery? 37

Q33. How do you refactor Terraform code (rename resources, move them into modules) without downtime or recreation? . . . 38

Q34. How do you perform zero-downtime updates with Terraform? . . . 39

Q35. What are circular dependencies in Terraform, and how do you fix them? . . . 40

Scenario-Based **41**

Q36. A resource was manually changed in AWS (for example someone added an S3 bucket policy to a Terraform-managed bucket), and Terraform shows drift. How do you handle it using IaC? . . . 41

Q37. There is an existing VPC created manually. How would you import it into Terraform without breaking anything? What are the prerequisites and how do you pass the VPC ID? . . . 42

Q38. Your Terraform state file was accidentally deleted, lost, or got corrupted / out of sync. How do you recover it safely? What are the risks and how do you protect it? . . . 43

Q39. A terraform apply fails midway in production after creating some resources. What would be your next steps? . . . 44

Q40. terraform plan shows that critical resources will be deleted unexpectedly. What would you do? . . . 45

Q41. A Terraform script bypassed validation and deleted a production database (or other prod resources) during peak business hours. What is your action plan, and how do you contain the blast radius? . . . 47

Q42. Terraform creates an IAM role and immediately uses it, but the apply fails even though the trust policy and permissions are correct. Why, and how do you fix it? . . . 48

Q43. Scenario: you need to create dynamic IAM roles across multiple environments. What is your module strategy? . . . 49

Q44. Scenario: AWS API rate limits (throttling) are hit during terraform apply. How do you handle it? . . . 51

Q45. Scenario: you added a new EC2 instance to an Auto Scaling Group using Terraform, but the ASG does not register the instance. How do you troubleshoot? . . . 52

Q46. What is the fastest way to roll back an infrastructure change made with Terraform? . . . 53

Q47. Why might terraform plan show NO changes even though someone modified the infrastructure outside Terraform? . . . 54

Q48. Terraform state is huge (200MB) and plan takes 12 minutes. How do you fix it? . . . 55

Terraform & Infrastructure as Code

Basic

Q1. What is Infrastructure as Code, and how do you manage it using Terraform or CloudFormation?

Also asked as: What are the advantages of IaC tools like Terraform?

> **In simple words:** Instead of clicking around the AWS console to build servers, you write a recipe file. Anyone can read the recipe, review it, and cook the exact same dish again and again.

Detailed answer

Infrastructure as Code (IaC) means your VPCs, servers, databases and DNS records are described in text files that live in Git. A tool reads those files and creates or changes the real cloud resources to match.

- **Terraform** (HashiCorp, or its open-source fork OpenTofu) works with almost any cloud or SaaS through "providers". It keeps a **state file** to remember what it built.
- **CloudFormation** is AWS-only. You write YAML/JSON "templates"; AWS itself stores the state inside a "stack".
- The basic Terraform workflow: `init` (download providers) -> `plan` (preview changes) -> `apply` (make changes) -> `destroy` (remove).
- Good practice: code in Git, pull requests for review, `plan` run automatically in CI, `apply` only after approval.

Advantages of IaC:

- **Repeatability**: the same code builds identical dev, stage and prod, with no snowflake setups.
- **Version control and review**: every change is a Git commit and a pull request, so you can see who, what and why, and roll back.
- **Drift detection**: `plan` shows when reality no longer matches the code.
- **Speed**: a whole environment in minutes instead of days of clicking.
- **DR rebuild**: re-create the stack in another region from code.
- **Documentation as code**: the code is the always-current description of the infrastructure.
- **Multi-cloud**: one tool and workflow for AWS, GCP, Azure and SaaS such as Datadog or GitHub.
- **Cost and policy checks in CI**: tools like Infracost, Checkov and OPA review a change before it exists.

Example:

```
provider "aws" {
  region = "ap-south-1"
}

resource "aws_s3_bucket" "logs" {
  bucket = "myapp-prod-logs"
  tags   = { Env = "prod", Owner = "devops" }
}
```

```
terraform init
terraform plan   # preview: 1 to add, 0 to change, 0 to destroy
terraform apply
```

> ***What to say to the interviewer:*** *IaC means defining infrastructure in version-controlled code instead of manual console clicks, so it is repeatable, reviewable and auditable. I use Terraform with modules and remote state, run plan in CI on every pull request, and apply only after review. CloudFormation is the AWS-native alternative where AWS manages state as stacks.*

Q2. What is the Terraform state file and why is it important?

Also asked as: Why is the state file important, and how does DynamoDB locking protect it?

Also asked as: Explain the state file and where you store it. What are the best practices for storing state?

> **In simple words:** The state file is Terraform's notebook. It writes down "I created this bucket and its real AWS ID is X". Without the notebook, Terraform forgets what it owns.

Detailed answer

`terraform.tfstate` is a JSON file that maps each resource in your code (for example `aws_instance.web`) to the real object in the cloud (for example `i-0abc123`). It also stores attributes like IPs, ARNs and dependencies.

Terraform uses it to:

- Know what already exists, so it updates instead of re-creating.
- Calculate the difference (the plan) between code, state and reality.
- Track dependencies so it deletes things in the right order.
- Speed up plans for large setups.

Important points:

- State can contain **secrets in plain text** (DB passwords, keys). Never commit it to Git.
- Store it remotely (S3, GCS, Azure Blob, HCP Terraform) with encryption, versioning and locking.
- Never hand-edit it. Use `terraform state list`, `state show`, `state mv`, `state rm`.

Example:

```
terraform state list
# aws_vpc.main
# aws_instance.web
terraform state show aws_instance.web   # shows id, private_ip, ami, ...
```

> ***What to say to the interviewer:*** *The state file is the mapping between my Terraform code and the real resources in the cloud, and Terraform uses it to compute plans and dependencies. Because it can hold secrets and is critical, I keep it in a remote encrypted and versioned backend like S3 with locking, never in Git, and I only change it using terraform state commands.*

Q3. What are the two main types of variables in Terraform? How are locals and outputs different?

In simple words: Input variables are the blanks in a form you fill in. Environment variables are another way of filling those blanks without typing. Locals are your own scratch calculations, and outputs are the answers you print at the end.

Detailed answer

1. **Input variables** - declared with a `variable` block and set via `terraform.tfvars`, `*.auto.tfvars`, `-var-file`, or `-var` on the command line. They can have a type, a default, a description, validation and `sensitive = true`.
2. **Environment variables (TF_VAR_name)** - Terraform automatically reads any env var named `TF_VAR_<variable_name>` into the matching input variable. Very useful in CI to inject secrets without writing them to files.

Other related concepts:

- **locals** - computed values inside a module (like naming prefixes). Cannot be set from outside.
- **outputs** - values a module exposes (like a VPC ID) to the user or to another module.

Precedence is explained in the next question (variables.tf vs .tfvars).

Example:

```
variable "instance_type" {
  type    = string
  default = "t3.micro"
}

variable "db_password" {
  type      = string
  sensitive = true
}

locals {
  name_prefix = "myapp-${terraform.workspace}"
}

output "web_ip" {
  value = aws_instance.web.private_ip
}
```

```
export TF_VAR_db_password="from-ci-secret-store"
terraform apply -var="instance_type=t3.small"
```

What to say to the interviewer: *The two main types are input variables declared with variable blocks and set through tfvars files or the command line, and environment variables using the TF_VAR_ prefix, which Terraform maps automatically. I use TF_VAR_ in pipelines for secrets, locals for derived values like name prefixes, and outputs to pass IDs between modules.*

Q4. What is the difference between count and for_each, and what is a for expression? When do you use each?

In simple words: count is like numbering seats 0, 1, 2 - if one person leaves, everyone shifts seats. for_each is like name tags - if one person leaves, nobody else moves.

Detailed answer

- **count** creates N copies of a resource, addressed by index: `aws_instance.web[0]`, `[1]`, `[2]`. Best for "N identical things" or an on/off switch (`count = var.enabled ? 1 : 0`).
- **The index-shift problem**: with `count = length(var.users)`, removing the first item from the list shifts every index down by one. Terraform then thinks every later item changed and plans to destroy and re-create them.
- **for_each** takes a map or a set of strings and addresses each copy by key: `aws_iam_user.u["alice"]`. Removing one key only removes that one resource. Use it whenever items have their own identity or different settings.
- **for expression** does not create resources. It transforms a list or map into a new list or map, often to build the input for for_each or an output.

Use	When
count	N identical copies, or a conditional resource
for_each	items with names or different settings (users, buckets, subnets)
for expression	reshape or filter data

Example:

```
variable "buckets" {
  default = {
    logs   = { versioning = true }
    assets = { versioning = false }
  }
}

resource "aws_s3_bucket" "b" {
  for_each = var.buckets
  bucket   = "acme-${each.key}"
  tags     = { Versioning = tostring(each.value.versioning) }
}

locals {
  names_upper = [for x in ["dev", "prod"] : upper(x)]         # ["DEV", "PROD"]
  bucket_arns = { for k, b in aws_s3_bucket.b : k => b.arn }  # map of key -> ARN
}
```

What to say to the interviewer: *count creates copies by numeric index, which is fine for identical resources or on/off switches, but removing an item shifts the indexes and can recreate resources. for_each keys each copy by a map key or set value, so changes stay isolated, and I prefer it for anything with an identity. A for expression like [for x in list : upper(x)] only transforms data, for example to build the map I pass into for_each.*

Q5. What is the difference between variables.tf and .tfvars files? What is the variable precedence order?

In simple words: variables.tf is the form with empty boxes and labels ("instance type: ____"). A .tfvars file is a filled-in copy of that form, one per environment.

Detailed answer

- **variables.tf** = **declaration**: name, type, description, default, validation, sensitive. It says what inputs exist.
- **.tfvars** = **values**: `terraform.tfvars` and `*.auto.tfvars` are loaded automatically; others like `prod.tfvars` are passed with `-var-file`.
- Same code, different tfvars per environment (dev.tfvars, prod.tfvars). Do not commit tfvars that contain secrets.

Precedence (highest wins):

Rank	Source
1 (highest)	`-var` on the command line
2	`-var-file=...`
3	`*.auto.tfvars` (alphabetical order)
4	`terraform.tfvars`
5	`TF_VAR_name` environment variable
6 (lowest)	`default` in the variable block

If a variable has no default and no value is given, Terraform asks for it interactively (or fails in CI with `-input=false`).

Example:

```
# variables.tf
variable "instance_type" {
  type        = string
  description = "EC2 size"
  default     = "t3.micro"
  validation {
    condition     = can(regex("^t3\\.", var.instance_type))
    error_message = "Only t3 family allowed."
  }
}
```

```
# prod.tfvars
instance_type = "t3.large"
```

```
terraform plan -var-file=prod.tfvars                      # t3.large
terraform plan -var-file=prod.tfvars -var="instance_type=t3.xlarge"   # -var wins
```

What to say to the interviewer: *variables.tf declares the inputs with their types, defaults and validation, while tfvars files supply the actual values, usually one file per environment. The precedence from highest to lowest is -var, then -var-file, then auto.tfvars, then terraform.tfvars, then TF_VAR_ environment variables, and finally the default.*

Q6. In which Terraform file or block can you identify the provider version (for example the GCP google provider)?

Also asked as: How do you handle provider API changes between provider versions?

In simple words: The terraform block is your shopping list saying "I need the google plugin, version 6 or newer". The lock file is the receipt saying exactly which version was actually bought.

Detailed answer

- **required_providers** inside the `terraform {}` block declares the provider source and the allowed version range. By convention it lives in `versions.tf` or `providers.tf`, but it can be in any `.tf` file.
- **.terraform.lock.hcl** records the exact version that `terraform init` selected, plus checksums. Commit this file so everyone and CI use the same version.
- The `provider "google" {}` block itself configures project/region; it should not normally contain the version.
- Commands: `terraform version` shows installed providers; `terraform providers` lists requirements; `terraform init -upgrade` moves to a newer allowed version.

Handling provider API changes between versions:

- Pin a range like `version = "~> 5.0"` so a new major version (with breaking changes) is never picked up by surprise.
- Commit `.terraform.lock.hcl` so every laptop and CI run uses the exact same build.
- Upgrade deliberately: `terraform init -upgrade` in a branch, read the provider upgrade guide and changelog, fix deprecated arguments, and run `plan` in staging before prod.
- For mixed Windows/Mac/Linux teams, record checksums for all platforms: `terraform providers lock -platform=linux_amd64 -platform=darwin_arm64 -platform=windows_amd64`.

Example:

```
# versions.tf
terraform {
  required_version = ">= 1.6"
  required_providers {
    google = {
      source  = "hashicorp/google"
      version = "~> 6.0"   # any 6.x, not 7.0
    }
  }
}
```

```
# .terraform.lock.hcl (generated)
provider "registry.terraform.io/hashicorp/google" {
  version     = "6.8.0"
  constraints = "~> 6.0"
  hashes      = [ "h1:..." ]
}
```

What to say to the interviewer: *The version constraint is declared in the required_providers section of the terraform block, usually in versions.tf, and the exact version actually installed is pinned in .terraform.lock.hcl, which I commit to Git. I can also check it with terraform version or terraform providers.*

Q7. What files are in a typical Terraform folder or module, and what is each file for?

In simple words: Terraform reads every .tf file in a folder as one big file. Splitting it into main, variables, outputs and so on is just tidy labelling, like drawers in a cupboard.

Detailed answer

File / folder	Purpose
`main.tf`	The main resources (and module calls)
`variables.tf`	Input variable declarations
`outputs.tf`	Values exposed to the user or to other modules
`providers.tf` / `versions.tf`	`terraform {}` block with required_version and required_providers, plus provider config
`backend.tf`	Remote state backend (S3 / GCS / azurerm) - only in root modules
`locals.tf`	Computed local values (name prefixes, common tags)
`data.tf`	Data sources (lookups of existing AMIs, VPCs, accounts)
`terraform.tfvars` / `prod.tfvars`	Values for variables, per environment
`.terraform/`	Downloaded providers and modules (created by init, never commit)
`.terraform.lock.hcl`	Exact provider versions and checksums (commit it)
`terraform.tfstate`	Exists locally only when using the local backend - avoid in teams
`.gitignore`	Ignores `.terraform/`, `*.tfstate*`, secret tfvars, crash logs

Root module vs child module: a **root module** (what you run `terraform apply` in) has backend and provider config; a **child module** (reusable, under `modules/`) has only resources, variables and outputs, and never its own backend. The module design question in the Advanced section covers versioning and DRY patterns.

Example:

```
infra/
  modules/
    vpc/
      main.tf  variables.tf  outputs.tf  versions.tf  README.md
    eks/
      main.tf  variables.tf  outputs.tf  versions.tf
  envs/
    dev/
      main.tf  backend.tf  providers.tf  dev.tfvars  .terraform.lock.hcl
    prod/
      main.tf  backend.tf  providers.tf  prod.tfvars  .terraform.lock.hcl
  .gitignore
```

```
# .gitignore
.terraform/
*.tfstate
*.tfstate.*
crash.log
secrets.auto.tfvars
```

What to say to the interviewer: *A typical folder has main.tf for resources, variables.tf for inputs, outputs.tf for outputs, versions.tf or providers.tf for provider requirements, backend.tf for remote state, plus locals.tf and data.tf, and tfvars files for per-environment values. .terraform is the downloaded plugin cache that I never commit, while .terraform.lock.hcl is committed, and child modules under modules/ contain only resources, variables and outputs.*

Q8. How does Terraform prevent duplicate resource creation?

In simple words: Before building anything, Terraform checks its notebook (state). If the notebook says "already built this", it does not build a second one, it only fixes differences.

Detailed answer

- Every resource has a unique **address** in code, like `aws_instance.web`. State maps that address to one real ID.
- On `plan`, Terraform refreshes state against the real cloud and compares with code. If the resource exists and matches, it does nothing (idempotent).
- **State locking** stops two people running apply at the same time, which would otherwise create two copies.
- Duplicates can still happen if state is lost, if two teams use different state files for the same thing, or if a resource was created manually. Fix: `import` it instead of creating it.
- Many cloud APIs also reject duplicates (S3 bucket names, IAM role names are unique).
- `count` / `for_each` give each copy a stable key so repeated runs do not add more.

Example:

```
terraform apply    # first run: 1 to add
terraform apply    # second run:
# No changes. Your infrastructure matches the configuration.
```

What to say to the interviewer: *Terraform is idempotent because it tracks every resource address against a real ID in the state file, so a second apply only shows differences instead of creating duplicates. Remote state with locking prevents concurrent runs, and for resources that already exist outside Terraform I use import rather than letting it create a copy.*

Q9. Write Terraform code to launch an EC2 instance.

In simple words: Tell Terraform four things: which cloud and region, which operating system image, what firewall rules, and what size of machine. Then preview and build.

Detailed answer

A minimal but complete setup has: a `terraform` block (provider version), a `provider` block (region), a data source to find the latest AMI, a security group, the `aws_instance`, and an output. For a version that also looks up an existing VPC and subnets with data sources, see the "dynamically retrieve VPC details" question in the Advanced section.

Example:

```
terraform {
  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 5.0"
    }
  }
}

provider "aws" {
  region = "ap-south-1"
}

data "aws_ami" "ubuntu" {
  most_recent = true
  owners      = ["099720109477"] # Canonical
  filter {
    name   = "name"
    values = ["ubuntu/images/hvm-ssd*/ubuntu-*-24.04-amd64-server-*"]
  }
}

resource "aws_security_group" "web" {
  name = "web-sg"
  ingress {
    from_port   = 80
    to_port     = 80
    protocol    = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }
  egress {
    from_port   = 0
    to_port     = 0
    protocol    = "-1"
    cidr_blocks = ["0.0.0.0/0"]
  }
}

resource "aws_instance" "web" {
  ami                    = data.aws_ami.ubuntu.id
  instance_type          = "t3.micro"
  vpc_security_group_ids = [aws_security_group.web.id]
  tags                   = { Name = "web-server" }
}

output "public_ip" {
  value = aws_instance.web.public_ip
}
```

```
terraform init      # download the AWS provider
terraform plan      # preview: 2 to add
terraform apply     # create SG + instance, prints public_ip
terraform destroy   # clean up when done
```

What to say to the interviewer: *I declare the AWS provider in a terraform block, set the region, look up the latest Ubuntu AMI with a data source, create a security group, and reference both in an aws_instance resource, then output the public IP. The workflow is terraform init, plan to review, apply to create, and destroy to clean up.*

Q10. What is drift in Terraform?

In simple words: Your recipe says the cake has 2 eggs, but someone secretly added a third egg to the real cake. The difference between the recipe and the real cake is drift.

Detailed answer

Drift is when the real infrastructure no longer matches what the Terraform code and state say. It usually happens because:

- Someone changed a resource manually in the console (opened a security group port, edited a bucket policy).
- Another tool or script modified it (autoscaling, a Lambda, another team's pipeline).
- A resource was deleted outside Terraform.

Why it matters: the next `terraform apply` may silently undo the manual fix, or fail, and your code is no longer the source of truth.

How you see it: `terraform plan` (or `terraform plan -refresh-only`) shows what changed outside Terraform, marked as "Objects have changed outside of Terraform".

Example:

```
Note: Objects have changed outside of Terraform

  # aws_security_group.web has changed
  ~ ingress = [
      + { from_port = 22, to_port = 22, cidr_blocks = ["0.0.0.0/0"] },
    ]
```

What to say to the interviewer: *Drift is the gap between the desired state in code and the actual state in the cloud, usually caused by manual console changes or other automation. I detect it with terraform plan or plan -refresh-only, ideally on a schedule in CI, and then either revert it by applying code or codify the change if it was intended.*

Q11. What is the use of the terraform refresh command?

In simple words: Refresh is Terraform re-reading the real world to update its notebook, without changing anything in the real world.

Detailed answer

- `terraform refresh` reads the current settings of every managed resource from the cloud and updates the state file to match reality. It does not change infrastructure.
- It is **deprecated** because it updates state silently with no review. The modern, safe way is `terraform plan -refresh-only` (preview) and `terraform apply -refresh-only` (accept the update after you review it).

- Normal `plan` and `apply` already do a refresh automatically first. You can skip that with `-refresh=false` for speed on huge setups.
- Use case: someone changed a resource manually and you want state to reflect it (then update code too), or you want to see drift without planning any changes.

Example:

```
terraform plan -refresh-only     # show what changed outside Terraform
terraform apply -refresh-only    # accept those changes into state only
```

What to say to the interviewer: *terraform refresh syncs the state file with the real infrastructure without modifying resources. It is deprecated now, so I use terraform plan -refresh-only to review drift and apply -refresh-only to accept it into state, because that gives a reviewable step instead of a silent state change.*

Q12. What is a null_resource in Terraform?

In simple words: A null_resource is an empty box that does not create anything in the cloud. You use it as a hook to run a script at the right moment.

Detailed answer

- `null_resource` (from the `hashicorp/null` provider) creates no real infrastructure. It is used together with **provisioners** (`local-exec`, `remote-exec`) to run commands, and with `triggers` to decide when to re-run.
- Typical uses: run a DB migration script after RDS is created, call an API, run `kubectl apply` once a cluster is up.
- Since Terraform 1.4, the built-in `terraform_data` resource does the same job without an extra provider, using `triggers_replace`.
- Caution: provisioners are a last resort. They are not idempotent and are hard to debug; prefer native resources, user_data, Ansible, or CI steps.

Example:

```
resource "terraform_data" "db_migrate" {
  triggers_replace = [aws_db_instance.app.id, var.schema_version]

  provisioner "local-exec" {
    command = "./migrate.sh ${aws_db_instance.app.address}"
  }
}
```

What to say to the interviewer: *A null_resource creates nothing in the cloud; it acts as a container for provisioners so I can run a script at a specific point in the dependency graph, re-run using triggers. In newer Terraform I use terraform_data instead, and I keep provisioners as a last resort because they are not declarative.*

Q13. What is the GCP (and Azure) equivalent of AWS CloudFormation?

In simple words: Every cloud has its own native recipe language. AWS has CloudFormation, Azure has ARM/Bicep, and Google has Deployment Manager and now Infrastructure Manager.

Detailed answer

Cloud	Native IaC	Notes
AWS	CloudFormation (plus CDK on top)	YAML/JSON templates, stacks
GCP	Deployment Manager (legacy) / Infrastructure Manager	Deployment Manager is deprecated; Infrastructure Manager is a managed service that runs Terraform
Azure	ARM templates / Bicep	Bicep is the friendlier language that compiles to ARM

Why many teams still pick Terraform: one language across all clouds, huge provider ecosystem, and on GCP Google itself recommends Terraform (via Infrastructure Manager).

Example:

```
# GCP Infrastructure Manager deploying a Terraform config from Git
gcloud infra-manager deployments apply projects/my-proj/locations/us-central1/deployments/net \
  --service-account=projects/my-proj/serviceAccounts/im-sa@my-proj.iam.gserviceaccount.com \
  --git-source-repo=https://github.com/org/infra --git-source-directory=network
```

> ***What to say to the interviewer:*** *The classic GCP equivalent is Deployment Manager, but Google has deprecated it and now recommends Infrastructure Manager, which is a managed way to run Terraform. On Azure the equivalent is ARM templates or Bicep. In multi-cloud setups I prefer Terraform because it is one workflow everywhere.*

Q14. What are some non-cloud-native alternatives to Terraform?

> **In simple words:** Terraform is one brand of recipe book. Others exist - some use real programming languages, some focus on configuring servers rather than creating them.

Detailed answer

- **OpenTofu** - open-source fork of Terraform (Linux Foundation), near drop-in compatible.
- **Pulumi** - IaC using real languages (TypeScript, Go, Python) with state like Terraform.
- **Crossplane** - manages cloud resources as Kubernetes custom resources (GitOps friendly).
- **Ansible** - mainly configuration management (install packages, configure servers), but it can create cloud resources too. Agentless, push-based.
- **Chef / Puppet / SaltStack** - configuration management, agent-based, pull model.
- **Terragrunt** - a wrapper around Terraform for DRY multi-environment setups (not a replacement).
- **AWS CDK / CDKTF** - write code that generates CloudFormation or Terraform.

Key distinction: **provisioning tools** (Terraform, Pulumi, Crossplane) create infrastructure; **configuration management tools** (Ansible, Chef, Puppet) configure what runs on it.

Example:

```
Terraform  -> create VPC, EC2, RDS
Ansible    -> install nginx, push config files, restart service on those EC2s
```

What to say to the interviewer: *Alternatives include OpenTofu, Pulumi, Crossplane for provisioning, and Ansible, Chef or Puppet for configuration management. A common pattern is Terraform to provision the infrastructure and Ansible to configure the servers, or baking images with Packer so no configuration is needed at boot.*

Q15. What is immutable infrastructure, and how do Packer and Terraform help?

In simple words: Instead of repairing an old car on the road, you build a fresh identical car in the factory and swap it in. Servers are replaced, never patched in place.

Detailed answer

- **Mutable**: SSH into servers and patch/upgrade them. Over time each server becomes slightly different ("snowflake servers") and failures are hard to reproduce.
- **Immutable**: bake a new machine image with everything installed, deploy new instances from it, and destroy the old ones.
- **Packer** builds the golden image (AMI) from a template, often using shell or Ansible provisioners.
- **Terraform** then points the Launch Template to the new AMI and an Auto Scaling Group instance refresh rolls it out.
- Benefits: consistent servers, easy rollback (previous AMI), faster boot, less drift, better security patching cadence.
- Containers are the same idea: build a new image tag, never change a running container.

Example:

```
# Packer (golden.pkr.hcl)
source "amazon-ebs" "base" {
  region        = "ap-south-1"
  instance_type = "t3.micro"
  source_ami    = "ami-0abc123"
  ssh_username  = "ubuntu"
  ami_name      = "web-{{timestamp}}"
}
build {
  sources = ["source.amazon-ebs.base"]
  provisioner "shell" { inline = ["sudo apt-get update", "sudo apt-get install -y nginx"] }
}
```

```
# Terraform picks the newest baked AMI
data "aws_ami" "web" {
  most_recent = true
  owners      = ["self"]
  filter {
    name   = "name"
    values = ["web-*"]
  }
}
```

What to say to the interviewer: *Immutable infrastructure means we replace servers instead of patching them. I bake a golden AMI with Packer, reference it in Terraform through a launch template, and roll it out with an ASG instance refresh, so every server is identical and rollback is just pointing back to the previous image.*

Advanced

Q16. How do you set up remote state and state locking, especially when multiple engineers modify infrastructure at the same time?

Also asked as: How do you avoid state conflicts? How do you lock state with S3 and DynamoDB?

Also asked as: Two engineers run terraform apply at the same time. What happens and how do you prevent it?

In simple words: Keep the notebook in a shared locker, not on anyone's laptop. When one person is writing in it, the locker is locked so nobody else can scribble at the same time.

Detailed answer

Remote state:

- Store state in a shared backend: **S3** (AWS), **GCS**, **Azure Blob**, or **HCP Terraform / Terraform Cloud**.
- Turn on **encryption** (SSE-KMS), **versioning** (to roll back a bad state), block public access, and restrict IAM access to the bucket.

Locking:

- While `plan`/`apply` runs, Terraform takes a lock. A second person gets `Error acquiring the state lock` and must wait.
- S3: since Terraform 1.10, **S3 native locking** (`use_lockfile = true`) is the recommended path - Terraform writes a `.tflock` object next to the state. The older **DynamoDB table** method (`dynamodb_table`, key `LockID`) still works on existing setups but is deprecated.
- GCS and HCP Terraform lock automatically.
- If a run crashes and leaves a stale lock, confirm nobody is running, then `terraform force-unlock <LOCK_ID>`.

Team process on top of locking:

- If two engineers apply at once with locking, the second one fails fast with `Error acquiring the state lock`. Without locking, both write state and one overwrites the other -> corrupted state, duplicates or orphans.
- Only CI applies (not laptops), one pipeline queue per state.
- Split big states by component/env so teams do not block each other.
- Plan on pull request, apply after merge (or tools like Atlantis).

Example:

```
terraform {
  backend "s3" {
    bucket       = "acme-tf-state"
    key          = "prod/network/terraform.tfstate"
    region       = "ap-south-1"
    encrypt      = true
    use_lockfile = true          # Terraform 1.10+ native S3 lock
    # dynamodb_table = "tf-locks" # older approach
  }
}
```

```
terraform force-unlock 6f1b2c3d-...   # only after confirming no run is active
```

> ***What to say to the interviewer:*** *I keep state in an encrypted, versioned S3 bucket with locking - native S3 lockfile on newer Terraform or a DynamoDB table on older setups - so only one apply can run at a time. On top of that, only CI applies, states are split per environment and component, and force-unlock is used only after confirming the lock is stale.*

Q17. How do you manage existing (unmanaged) AWS resources in Terraform? What is the difference between terraform import and an import block?

Also asked as: How do you bring manually created resources under Terraform?

> **In simple words:** The resource already exists; you just need to write it into Terraform's notebook and write matching code, so Terraform adopts it instead of building a duplicate.

Detailed answer

Two ways to adopt existing resources:

	terraform import (CLI)	import block (Terraform 1.5+)
Where	One-off command	Written in .tf code
Reviewable in PR / plan	No, changes state immediately	Yes, shows in plan before apply
Many resources at once	One per command	Many blocks, also works with for_each
Generates config	No, you write code by hand	Yes, `-generate-config-out`

Steps (import block way):

1. Write an `import` block with the target address and the real ID.
2. Run `terraform plan -generate-config-out=generated.tf` to auto-write resource code.
3. Clean up the generated code (remove computed/default noise, use variables).
4. Run `plan` until it shows `1 to import, 0 to add, 0 to change, 0 to destroy`.
5. Apply, then you can delete the import block.

Other adoption tools: `terraformer` or `aws2tf` for bulk reverse-engineering.

Example:

```
import {
  to = aws_s3_bucket.assets
  id = "acme-prod-assets"
}
```

```
terraform plan -generate-config-out=generated.tf
# Old CLI way:
terraform import aws_s3_bucket.assets acme-prod-assets
```

> ***What to say to the interviewer:*** *To manage unmanaged resources I write matching Terraform code and import them into state. The old terraform import command changes state immediately and one resource at a time, while the import block from Terraform 1.5 is declarative, shows up in the plan for review, and can even generate config, so I prefer the import block.*

Q18. How do you dynamically retrieve VPC details from AWS to create an EC2 instance using IaC? Write the code.

In simple words: Instead of hard-coding "subnet-123" in your recipe, you ask AWS "give me the VPC tagged prod and its private subnets" at run time. That question is a data source.

Detailed answer

- **Data sources** (`data` blocks) read existing information without managing it.
- Look up the VPC by tag, then its subnets, then the latest AMI, and feed those into the EC2 resource.
- This makes the same code work in every account/environment and avoids breaking when IDs change.

Example:

```
data "aws_vpc" "main" {
  filter {
    name   = "tag:Name"
    values = ["prod-vpc"]
  }
}

data "aws_subnets" "private" {
  filter {
    name   = "vpc-id"
    values = [data.aws_vpc.main.id]
  }
  tags = { Tier = "private" }
}

data "aws_ami" "al2023" {
  most_recent = true
  owners      = ["amazon"]
  filter {
    name   = "name"
    values = ["al2023-ami-*-x86_64"]
  }
}

resource "aws_security_group" "app" {
  name   = "app-sg"
  vpc_id = data.aws_vpc.main.id
}

resource "aws_instance" "app" {
  ami                    = data.aws_ami.al2023.id
  instance_type          = "t3.micro"
  subnet_id              = data.aws_subnets.private.ids[0]
  vpc_security_group_ids = [aws_security_group.app.id]
  tags                   = { Name = "app-server" }
}
```

If the VPC is managed by another Terraform stack, you can instead read its outputs with `data "terraform_remote_state"`, or better, SSM parameters.

What to say to the interviewer: *I use data sources - aws_vpc filtered by tag, aws_subnets filtered by VPC ID and tier, and aws_ami for the latest image - and pass their IDs into the aws_instance resource. That way nothing is hard-coded and the same code works across environments.*

Q19. If you want to replace (recreate) a single resource in existing infrastructure, how would you do it?

In simple words: One brick in the wall is bad. You tell Terraform "replace just this brick", not rebuild the whole wall.

Detailed answer

- Modern way: `terraform apply -replace="ADDRESS"` - forces destroy-and-recreate of only that resource in one reviewed plan.
- Old way: `terraform taint ADDRESS` then apply. `taint` is deprecated because it changes state before you see a plan.
- Only touch one resource and its dependents: `-target=ADDRESS` (for emergencies only, not daily use).
- To avoid downtime while replacing, use `lifecycle { create_before_destroy = true }`.
- To make Terraform always replace when something changes: `lifecycle { replace_triggered_by = [...] }`.

Example:

```
terraform plan  -replace="aws_instance.web[1]"
terraform apply -replace="aws_instance.web[1]"
# Plan: 1 to add, 0 to change, 1 to destroy.
```

```
resource "aws_instance" "web" {
  # ...
  lifecycle {
    create_before_destroy = true
  }
}
```

What to say to the interviewer: *I use terraform apply -replace with the resource address, which forces recreation of only that resource while showing me the plan first; taint is the older, deprecated way. If downtime matters I add create_before_destroy so the new one comes up before the old one is removed.*

Q20. How do you control precisely which resources are destroyed and which are kept?

Also asked as: Scenario: a resource was removed from the config but is still live. What happens and how do you handle it?

In simple words: You want to demolish the garage but keep the house. Terraform gives you tools to point at exactly one thing, to put a "do not demolish" sign on others, or to simply stop looking after something without knocking it down.

Detailed answer

Goal	Tool
Destroy only one resource	`terraform destroy -target=ADDRESS` (or remove it from code and apply) - use carefully
Never allow a resource to be destroyed by Terraform	`lifecycle { prevent_destroy = true }` - plan errors out
Stop managing a resource but keep it in the cloud	`terraform state rm ADDRESS`, or the `removed` block (TF 1.7+) with `destroy = false`
Protect at the cloud level too	`deletion_protection = true` (RDS, ALB), S3 object lock, termination protection
Limit blast radius	split state: databases and networking in their own root modules
Review before destroy	`terraform plan -destroy -out=destroy.plan` then apply that plan

The `removed` block is the reviewable, code-based way to "forget" a resource: the plan shows it will be removed from state, not destroyed.

Resource removed from config but still live:

- Common misconception: deleting a resource block does not make Terraform "forget" it. The resource is still in state, so the next `plan` shows it as **to destroy**, and apply deletes it.
- To keep it running but stop managing it: add a `removed` block with `destroy = false` (TF 1.7+) or run `terraform state rm`.
- `terraform apply -refresh-only` never destroys anything; it only updates state to match reality.
- Truly **orphaned** resources (live in the cloud but in no state file) come from `state rm`, failed applies that did not record state, or manual creation. Find them with tag-based inventories (e.g. every managed resource tagged `ManagedBy=terraform` + stack name), AWS Config / Resource Explorer, or driftctl-style tooling, then either import them or delete them.

Example:

```
removed {
  from = aws_instance.legacy_batch
  lifecycle {
    destroy = false   # remove from state, keep the real EC2
  }
}

resource "aws_db_instance" "main" {
  # ...
  deletion_protection = true
  lifecycle {
    prevent_destroy = true
  }
}
```

```
terraform plan -destroy -target=aws_instance.test_box -out=destroy.plan
terraform apply destroy.plan
terraform state rm aws_s3_bucket.shared_logs   # CLI way to stop managing it
```

What to say to the interviewer: *To destroy a single resource I remove it from code or use destroy with -target and review the saved plan. To keep resources, I use prevent_destroy and cloud-level deletion_protection, and when I want Terraform to stop managing something without deleting it I use a removed block with destroy = false or terraform state rm. Splitting state so databases live separately limits the blast radius even further.*

Q21. Besides depends_on, how do you control the order in which resources are deployed? Can you add a time delay?

In simple words: Terraform builds things in parallel unless one thing clearly needs another. You can show the need by pointing to it, add an explicit "wait for X", or even put in a "wait 30 seconds" timer.

Detailed answer

Ways to control order, from best to last resort:

1. **Implicit dependency (preferred)**: reference an attribute, e.g. `subnet_id = aws_subnet.a.id`. Terraform builds the graph automatically.
2. **depends_on**: for hidden dependencies Terraform cannot see (an IAM policy that must be attached before an app starts).
3. **time_sleep** (hashicorp/time provider): adds a delay after a resource is created or before it is destroyed. Useful for eventual consistency (IAM, DNS).
4. **terraform_data / null_resource with triggers**: run a `local-exec` wait loop, for example poll an endpoint until it returns 200.
5. **Separate root modules / pipeline stages**: network stack -> cluster stack -> apps stack. The strongest ordering, and it reduces blast radius.
6. `-target`: emergency tool only, not for routine ordering.
7. Provisioners in general: last resort, not declarative.

Example:

```
resource "aws_iam_role_policy_attachment" "app" {
  role       = aws_iam_role.app.name
  policy_arn = aws_iam_policy.app.arn
}

resource "time_sleep" "wait_iam" {
  depends_on      = [aws_iam_role_policy_attachment.app]
  create_duration = "20s"
}

resource "aws_lambda_function" "app" {
  depends_on    = [time_sleep.wait_iam]
  function_name = "app"
  role          = aws_iam_role.app.arn      # implicit dependency on the role
  runtime       = "python3.12"
  handler       = "app.handler"
  filename      = "app.zip"
}

resource "terraform_data" "wait_for_api" {
  triggers_replace = [aws_lb.api.dns_name]
  provisioner "local-exec" {
    command = "until curl -sf https://${aws_lb.api.dns_name}/health; do sleep 10; done"
  }
}
```

What to say to the interviewer: *I rely first on implicit dependencies through attribute references, use depends_on only for hidden dependencies, and for timing issues like IAM propagation I add a time_sleep resource with a create_duration. For waiting on something to become ready I can use terraform_data with a local-exec wait loop, and for strong ordering I split infrastructure into separate stacks applied in sequence.*

Q22. How do you design Terraform modules and keep code DRY?

Also asked as: How do you create and use Terraform modules?

In simple words: A module is a reusable Lego kit. Build a "VPC kit" once, then snap it into dev, stage and prod with different sizes instead of rewriting it three times.

Detailed answer

(For the list of files in a module folder, see the Basic question on Terraform folder files.)

Good module design:

- **Small and focused**: `vpc`, `eks`, `rds`, `alb` - not one giant "everything" module.
- **Clear interface**: `variables.tf` (inputs with types, descriptions, validation), `outputs.tf`, `main.tf`, `versions.tf`, `README.md`.
- **Sensible defaults**, but no hard-coded env names, account IDs, or regions.
- **Version the modules**: publish via Git tags or a private registry and pin `?ref=v1.4.0` so prod does not change when someone edits the module.
- **Compose** modules in thin "root" configs per environment.
- Use `for_each` / `count` and `locals` to avoid copy-paste.
- Test modules: `terraform validate`, `tflint`, `terraform test`, Checkov.
- Tools like **Terragrunt** remove repeated backend/provider blocks across many envs.

Example:

```
modules/
  vpc/ (main.tf variables.tf outputs.tf versions.tf)
  rds/
envs/
  dev/main.tf
  prod/main.tf
```

```
# envs/prod/main.tf
module "vpc" {
  source     = "git::https://github.com/acme/tf-modules.git//vpc?ref=v1.4.0"
  name       = "prod"
  cidr_block = "10.10.0.0/16"
  az_count   = 3
}
```

> ***What to say to the interviewer:*** *I build small, single-purpose modules with typed inputs and clear outputs, version them with Git tags, and compose them in thin root configurations per environment. Loops, locals and tools like Terragrunt keep it DRY, and every module is validated with tflint, Checkov and terraform test in CI.*

Q23. How do you manage separate environments (Dev, Test, Prod) in Terraform? Workspaces or separate directories?

Also asked as: How do you manage Dev, Staging and Prod with Terraform?

> **In simple words:** Dev and prod should have separate notebooks and separate lockers. Workspaces give separate notebooks in the same locker; separate folders or accounts give completely separate lockers.

Detailed answer

Options:

1. **Workspaces** - same code, separate state per workspace (`terraform workspace new prod`). Easy, but all envs share the same backend and credentials, and it is easy to apply to the wrong one. Good for short-lived or identical copies.
2. **Directory per environment** (most common in production) - `envs/dev`, `envs/prod`, each with its own backend key, tfvars, and provider credentials, all calling the same versioned modules.
3. **Terragrunt** - directory layout with DRY config.
4. **Separate AWS accounts per environment** (best isolation), combined with option 2.

Best practice:

- Different state files AND ideally different cloud accounts for prod.
- Same modules, different inputs (`dev.tfvars` small instances, `prod.tfvars` multi-AZ).
- Promotion: change is tested in dev, then the same module version is bumped in stage, then prod.
- Prod applies need approval in the pipeline.

Example:

```
envs/
  dev/   backend key: dev/terraform.tfstate   -> account 111111111111
  stage/ backend key: stage/terraform.tfstate -> account 222222222222
  prod/  backend key: prod/terraform.tfstate  -> account 333333333333
```

```
cd envs/prod && terraform init && terraform plan -var-file=prod.tfvars
```

> ***What to say to the interviewer:*** *I use one directory per environment, each with its own backend state, tfvars, and ideally its own AWS account, all reusing the same versioned modules. Workspaces are fine for quick identical copies, but for prod I want strong isolation and a promotion flow where the same module version moves from dev to stage to prod with approvals.*

Q24. How would you create resources across two different AWS accounts using Terraform?

> **In simple words:** You give Terraform two keycards - one for account A and one for account B - and label each resource with which keycard to use.

Detailed answer

- Define the `aws` provider twice using an **alias**.
- Each provider **assumes an IAM role** in its target account (no long-lived keys). The CI runner has base credentials only allowed to `sts:AssumeRole` into those roles.
- Point each resource or module to the right provider with `provider = aws.xxx` (or `providers = {}` for modules).
- Typical use cases: VPC peering between accounts, Route 53 records in a shared DNS account, cross-account S3 replication.

Example:

```
provider "aws" {
  alias  = "network"
  region = "ap-south-1"
  assume_role {
    role_arn = "arn:aws:iam::111111111111:role/TerraformDeploy"
  }
}

provider "aws" {
  alias  = "app"
  region = "ap-south-1"
  assume_role {
    role_arn = "arn:aws:iam::222222222222:role/TerraformDeploy"
  }
}

resource "aws_route53_record" "api" {
  provider = aws.network
  zone_id  = var.zone_id
  name     = "api.example.com"
  type     = "CNAME"
  ttl      = 300
  records  = [module.alb.dns_name]
}

module "alb" {
  source    = "./modules/alb"
  providers = { aws = aws.app }
}
```

> ***What to say to the interviewer:*** *I define two aws providers with aliases, each using assume_role into a deploy role in its account, and then set provider on each resource or pass providers into modules. The pipeline only holds credentials that can assume those roles, so there are no long-lived keys and every action is audited per account.*

Q25. How do you manage sensitive values like secrets or API keys in Terraform without hardcoding them?

Also asked as: How do you pass secrets into Terraform safely?

> **In simple words:** Never write the password in the recipe. Keep it in a safe (Secrets Manager / Vault) and let Terraform or the app open the safe at run time.

Detailed answer

Options, from good to best:

- Mark variables `sensitive = true` so they are hidden in plan output (they are still stored in state).
- Inject via CI secrets as `TF_VAR_db_password`, never in `.tfvars` committed to Git.
- Read from a secret store with a data source: AWS Secrets Manager / SSM Parameter Store, Vault, GCP Secret Manager.
- Let the cloud generate and hold the secret: e.g. RDS `manage_master_user_password = true` stores the password in Secrets Manager and it never appears in your code.
- Use `random_password` to generate, and store it straight into Secrets Manager.

- Newer Terraform (1.10+) adds ephemeral values and write-only arguments so some secrets never land in state at all.
- Always: encrypted, access-restricted remote state; `.gitignore` for `*.tfstate` and `*.tfvars`; secret scanning (gitleaks) in CI.

Example:

```
resource "aws_db_instance" "app" {
  identifier                  = "app-db"
  engine                      = "postgres"
  instance_class              = "db.t3.medium"
  allocated_storage           = 50
  username                    = "appadmin"
  manage_master_user_password = true   # password lives in Secrets Manager
}

data "aws_secretsmanager_secret_version" "api_key" {
  secret_id = "prod/payment/api-key"
}
```

> ***What to say to the interviewer:*** *I never hardcode secrets; I either inject them from CI as TF_VAR_ variables marked sensitive, read them from Secrets Manager or Vault with data sources, or better, let AWS generate and manage them like RDS managed master passwords. Because state can still contain secrets, the state bucket is encrypted with KMS and tightly access-controlled.*

Q26. How do you upgrade a GKE node pool version using Terraform (and manually)?

> **In simple words:** First upgrade the brain (control plane), then swap the workers (nodes) a few at a time, so the shop never closes.

Detailed answer

Order: control plane first, then node pools (nodes can be up to a couple of minor versions behind the master, never ahead).

Terraform way:

1. Bump `min_master_version` on `google_container_cluster` (or let a release channel handle it) and apply.
2. Bump `version` on `google_container_node_pool`, set `upgrade_settings` for surge upgrades, and apply. GKE drains and replaces nodes gradually.
3. For big jumps, use blue/green: create a new node pool with the new version, cordon/drain the old one, then remove it from code.
4. Many teams instead set `release_channel` + `auto_upgrade = true` with a maintenance window, and add `lifecycle { ignore_changes = [version] }` so Terraform does not fight auto-upgrades.

Manual way:

```
gcloud container clusters upgrade my-cluster --master --cluster-version=1.30 --region=asia-south1
gcloud container clusters upgrade my-cluster --node-pool=default-pool \
  --cluster-version=1.30 --region=asia-south1
```

Example:

```
resource "google_container_node_pool" "apps" {
  name     = "apps"
  cluster  = google_container_cluster.main.name
  location = "asia-south1"
  version  = "1.30.5-gke.1014001"

  upgrade_settings {
    max_surge       = 1
    max_unavailable = 0
  }

  management {
    auto_repair  = true
    auto_upgrade = false
  }
}
```

What to say to the interviewer: *I upgrade the control plane first, then change the version field on the google_container_node_pool resource with surge settings of max_surge 1 and max_unavailable 0, so nodes are replaced gradually and PodDisruptionBudgets keep apps up. For risky jumps I create a new node pool and drain the old one, and if we use release channels I ignore the version field so Terraform does not fight auto-upgrades.*

Q27. How do you create an EKS cluster using Terraform, and where do Kubernetes core components come into play?

Also asked as: When an EKS cluster is deployed via Terraform, what gets created? What is Karpenter used for?

In simple words: Terraform orders the building (network, cluster, worker nodes, permissions). AWS runs the brain of Kubernetes for you; your nodes are the workers.

Detailed answer

What Terraform creates (usually with the community `terraform-aws-modules/eks` module):

- VPC with private subnets for nodes and public subnets for load balancers (tagged for EKS).
- EKS cluster -> AWS runs and patches the **control plane** (API server, etcd, scheduler, controller manager) across 3 AZs.
- Managed node groups (EC2 in an Auto Scaling Group) -> these run **kubelet, kube-proxy, container runtime**.
- Add-ons: VPC CNI, CoreDNS, kube-proxy, EBS CSI driver.
- IAM: cluster role, node role, IRSA / Pod Identity for app permissions, access entries for who can use kubectl.

Also created: the **cluster security group** (control plane <-> nodes), the **OIDC issuer** URL (plus the IAM OIDC provider used by IRSA), **launch templates** for node groups, and access entries. Locally you get `terraform.tfstate` in the remote backend with its lock, and `.terraform.lock.hcl`.

The **kubeconfig** is not created by Terraform: you generate it with `aws eks update-kubeconfig`. Then deploy apps with Helm or Argo CD (keep app deployments separate from cluster Terraform).

Karpenter (short version): a node autoscaler that watches pending pods and launches right-sized EC2 instances directly in seconds, picks from many instance types including Spot, and consolidates

under-used nodes to save cost. Cluster Autoscaler only resizes pre-defined node groups, so it is slower and less flexible. The Kubernetes chapter covers it in depth.

Example:

```
module "eks" {
  source          = "terraform-aws-modules/eks/aws"
  version         = "~> 20.0"
  cluster_name    = "prod-eks"
  cluster_version = "1.30"
  vpc_id          = module.vpc.vpc_id
  subnet_ids      = module.vpc.private_subnets

  eks_managed_node_groups = {
    general = {
      instance_types = ["m6i.large"]
      min_size       = 2
      max_size       = 6
      desired_size   = 3
    }
  }
}
```

```
aws eks update-kubeconfig --name prod-eks --region ap-south-1
kubectl get nodes
```

> ***What to say to the interviewer:*** *I use the terraform-aws-modules EKS module on top of a VPC module; it creates the managed control plane, which AWS runs with the API server, etcd, scheduler and controllers, plus managed node groups that run kubelet and kube-proxy, and the core add-ons like VPC CNI and CoreDNS. Application deployment is kept separate, via Helm or Argo CD.*

Q28. You wrote Terraform for an EKS cluster plus a Kubernetes service account. The IAM role must exist before the cluster, and IRSA needs the OIDC provider, which only exists after the cluster. How do you solve the ordering?

> **In simple words:** It is a chain of dominoes. Terraform knows the order automatically if each piece points at the one before it, so you do not have to force anything.

Detailed answer

The correct chain:

1. **Cluster IAM role** (trusts `eks.amazonaws.com`) + policy attachment.
2. **EKS cluster** with `role_arn = aws_iam_role.cluster.arn` -> implicit dependency, the role is created first.
3. **IAM OIDC provider** using the cluster's issuer URL `aws_eks_cluster.this.identity[0].oidc[0].issuer` -> automatically created after the cluster.
4. **IRSA role** whose trust policy allows `sts:AssumeRoleWithWebIdentity` from that OIDC provider, with a condition on `<issuer>:sub = system:serviceaccount:<ns>:<sa>` (and `:aud = sts.amazonaws.com`).
5. **kubernetes_service_account** annotated with `eks.amazonaws.com/role-arn = <IRSA role ARN>`.

Key points:

- No `depends_on` needed: attribute references create the dependencies.
- Who can use kubectl: use **EKS access entries** (`aws_eks_access_entry` + access policy) instead of editing the old `aws-auth` ConfigMap; access entries are plain AWS API resources, so there is no chicken-and-egg.
- The **kubernetes provider** must be configured from cluster outputs (endpoint, CA, token). Providers cannot fully depend on resources in the same apply, so it is recommended to apply the cluster in one stack and Kubernetes objects (service accounts, Helm) in a separate apply/stack.
- Newer alternative: **EKS Pod Identity** (`aws_eks_pod_identity_association`) avoids the OIDC provider step entirely.

Example:

```
resource "aws_eks_cluster" "this" {
  name     = "prod-eks"
  role_arn = aws_iam_role.cluster.arn              # step 1 -> 2
  vpc_config { subnet_ids = var.private_subnets }
  access_config { authentication_mode = "API" }
}

data "tls_certificate" "oidc" {
  url = aws_eks_cluster.this.identity[0].oidc[0].issuer
}

resource "aws_iam_openid_connect_provider" "eks" {   # step 3
  url             = aws_eks_cluster.this.identity[0].oidc[0].issuer
  client_id_list  = ["sts.amazonaws.com"]
  thumbprint_list = [data.tls_certificate.oidc.certificates[0].sha1_fingerprint]
}

locals {
  issuer = replace(aws_iam_openid_connect_provider.eks.url, "https://", "")
}

data "aws_iam_policy_document" "irsa_trust" {
  statement {
    actions = ["sts:AssumeRoleWithWebIdentity"]
    principals {
      type        = "Federated"
      identifiers = [aws_iam_openid_connect_provider.eks.arn]
    }
    condition {
      test     = "StringEquals"
      variable = "${local.issuer}:sub"
      values   = ["system:serviceaccount:app:app-sa"]
    }
    condition {
      test     = "StringEquals"
      variable = "${local.issuer}:aud"
      values   = ["sts.amazonaws.com"]
    }
  }
}

resource "aws_iam_role" "app_irsa" {               # step 4
  name               = "prod-app-irsa"
  assume_role_policy = data.aws_iam_policy_document.irsa_trust.json
}

resource "kubernetes_service_account" "app" {      # step 5 (ideally separate stack)
  metadata {
    name        = "app-sa"
    namespace   = "app"
    annotations = { "eks.amazonaws.com/role-arn" = aws_iam_role.app_irsa.arn }
  }
}
```

What to say to the interviewer: *I let Terraform order it through attribute references: the cluster references the cluster role ARN, the OIDC provider references the cluster issuer URL, the IRSA role trusts that OIDC provider with a sub condition for the service account, and the service account is annotated with the role ARN. I use EKS access entries instead of aws-auth, and I configure the kubernetes provider from cluster outputs in a separate stack, or use EKS Pod Identity to skip the OIDC step.*

Q29. Write Terraform code to deploy an EKS cluster, a Helm chart and the Datadog agent, and use the agent for monitoring.

In simple words: Terraform builds the cluster, then installs the Datadog "spy" on every node using Helm, then creates an alarm rule in Datadog - all from one codebase.

Detailed answer

Pieces:

- `terraform-aws-modules/eks` creates the cluster and nodes.
- The **helm provider** authenticates to the cluster using its endpoint, CA and `aws eks get-token`.
- `helm_release` installs the `datadog/datadog` chart; the API key comes from a **sensitive** variable (fed from CI secrets / Secrets Manager) via `set_sensitive`.
- Values enable **logs, APM and the process agent**.
- The **Datadog provider** creates a `datadog_monitor` so alerts are code too.
- In real setups, put the Helm and Datadog parts in a separate stack from the cluster (same provider-ordering reason as IRSA).

Example:

```
terraform {
  required_providers {
    aws     = { source = "hashicorp/aws", version = "~> 5.0" }
    helm    = { source = "hashicorp/helm", version = "~> 2.17" }
    datadog = { source = "DataDog/datadog", version = "~> 3.0" }
  }
}

variable "datadog_api_key" {
  type      = string
  sensitive = true
}
variable "datadog_app_key" {
  type      = string
  sensitive = true
}

module "eks" {
  source          = "terraform-aws-modules/eks/aws"
  version         = "~> 20.0"
  cluster_name    = "prod-eks"
  cluster_version = "1.30"
  vpc_id          = var.vpc_id
  subnet_ids      = var.private_subnets
  enable_cluster_creator_admin_permissions = true
  eks_managed_node_groups = {
    general = { instance_types = ["m6i.large"], min_size = 2, max_size = 5, desired_size = 3 }
  }
}

provider "helm" {
  kubernetes {   # helm provider 3.x uses: kubernetes = { ... }
    host                   = module.eks.cluster_endpoint
    cluster_ca_certificate = base64decode(module.eks.cluster_certificate_authority_data)
    exec {
      api_version = "client.authentication.k8s.io/v1beta1"
      command     = "aws"
      args        = ["eks", "get-token", "--cluster-name", module.eks.cluster_name]
    }
  }
}

resource "helm_release" "datadog" {
  name             = "datadog"
  repository       = "https://helm.datadoghq.com"
  chart            = "datadog"
  namespace        = "datadog"
  create_namespace = true

  values = [yamlencode({
```

```
    datadog = {
      site          = "datadoghq.com"
      clusterName   = module.eks.cluster_name
      logs          = { enabled = true, containerCollectAll = true }
      apm           = { portEnabled = true }
      processAgent  = { enabled = true, processCollection = true }
    }
  })]

  set_sensitive {
    name  = "datadog.apiKey"
    value = var.datadog_api_key
  }
}

provider "datadog" {
  api_key = var.datadog_api_key
  app_key = var.datadog_app_key
}

resource "datadog_monitor" "pod_restarts" {
  name    = "[prod-eks] Pods restarting"
  type    = "query alert"
  query   = "change(max(last_5m),last_5m):sum:kubernetes_state.container.restarts{kube_cluster_name:
    prod-eks} by {kube_deployment} > 3"
  message = "Pods in {{kube_deployment.name}} are restarting. @teams-devops-alerts"
  monitor_thresholds {
    critical = 3
  }
}
```

```
export TF_VAR_datadog_api_key=... TF_VAR_datadog_app_key=...   # from CI secret store
terraform init && terraform plan -out=tfplan && terraform apply tfplan
kubectl get pods -n datadog    # agent DaemonSet + cluster agent
```

> ***What to say to the interviewer:*** *I create the cluster with the EKS module, configure the helm provider from the cluster endpoint, CA and aws eks get-token, and install the datadog chart with helm_release, passing the API key as a sensitive variable with set_sensitive and enabling logs, APM and process collection in the values. Then I use the Datadog provider to create monitors like a pod-restart alert, so the monitoring itself is code, and I keep the Helm layer in a separate stack from the cluster.*

Q30. How do you detect drift in Terraform and troubleshoot it?

> **In simple words:** Every night a guard compares the recipe with the real cake and raises a flag if anything differs.

Detailed answer

Detect:

- `terraform plan -refresh-only` - shows only what changed outside Terraform.
- `terraform plan -detailed-exitcode` in a scheduled CI job: exit code `0` = no changes, 2 = drift/changes, 1 = error. Alert on 2 (Teams/Slack).
- Managed drift detection: HCP Terraform, Spacelift, env0. (driftctl was used earlier but is no longer actively developed.)

- AWS Config / CloudTrail show who changed what and when.

Troubleshoot:

1. Read the plan diff: which resource and which attribute.
2. Check CloudTrail for the API call, user, and time.
3. Decide: was it an unwanted change (revert) or an intended hotfix (codify)?
4. Revert -> `terraform apply`. Codify -> update code until plan is clean, or `apply -refresh-only` then update code.
5. If a field is legitimately changed by something else (e.g. ASG desired count by autoscaler), add `lifecycle { ignore_changes = [...] }`.
6. Prevent: restrict console write access in prod (read-only roles, SCPs), all changes through PRs.

Example:

```
terraform plan -refresh-only -detailed-exitcode -input=false
echo $?   # 2 means drift found -> send alert
```

> ***What to say to the interviewer:*** *I run terraform plan with detailed-exitcode on a schedule in CI and alert when it returns 2, and I use CloudTrail to find who made the change. Then I either revert by applying the code or codify the change, use ignore_changes for fields managed by other systems, and prevent recurrence by making prod console access read-only.*

Q31. How do you integrate Terraform with CI/CD and safely deploy infrastructure changes to production?

> **In simple words:** Nobody changes production by hand. A robot shows everyone the exact plan, a human approves it, and the robot applies exactly that approved plan.

Detailed answer

Pipeline stages:

1. **Format and lint**: `terraform fmt -check`, `terraform validate`, `tflint`.
2. **Security / policy scan**: Checkov, tfsec/Trivy, and policy-as-code (OPA/Conftest or Sentinel) - e.g. "no public S3", "no deleting RDS".
3. **Plan** on every pull request; post the plan as a PR comment. Save it with `-out=tfplan`.
4. **Review and approval**: required reviewers plus a manual approval gate for prod (Jenkins input, GitHub environments).
5. **Apply the saved plan**: `terraform apply tfplan` - guarantees what was approved is what runs.
6. **Promote**: dev -> stage -> prod, same module version.
7. **Post-apply checks**: smoke tests, monitoring, and a scheduled drift check.

Safety guardrails:

- CI uses short-lived credentials (OIDC to an IAM role), no static keys.
- Fail the pipeline if the plan contains destroys of protected resource types unless explicitly approved.

- `prevent_destroy` and cloud-side `deletion_protection` on critical resources.
- Small, frequent changes; separate states per component to reduce blast radius.
- Schedule risky changes in a maintenance window with a rollback plan.

Example:

```
terraform fmt -check && terraform validate && tflint
checkov -d .
terraform plan -out=tfplan -input=false
terraform show -json tfplan > plan.json
conftest test plan.json          # OPA policies, e.g. deny destroy of aws_db_instance
# ... manual approval ...
terraform apply -input=false tfplan
```

> ***What to say to the interviewer:*** *My Terraform pipeline runs fmt, validate, tflint and Checkov, then generates a saved plan on every pull request, checks it against OPA policies such as no destroys on databases, and requires approval for prod. After approval it applies exactly that saved plan using OIDC short-lived credentials, and changes are promoted from dev to stage to prod.*

Q32. What are RPO and RTO, and how does Terraform / IaC help you meet them in disaster recovery?

Also asked as: Have you worked on DR? What is RTO vs RPO?

> **In simple words:** RPO is how much data you can afford to lose (for example "at most 5 minutes of orders"). RTO is how fast you must be back up (for example "running again within 1 hour"). Backups protect the RPO; being able to rebuild quickly protects the RTO.

Detailed answer

- **RPO (Recovery Point Objective)** - maximum acceptable data loss, measured in time.
- **RTO (Recovery Time Objective)** - maximum acceptable downtime until service is restored.

RPO - Terraform does not back up your data. Data RPO comes from database backups, point-in-time recovery (PITR), and cross-region replication. Terraform's job is to **configure** those consistently:

- `backup_retention_period` on RDS (enables automated backups + PITR).
- `aws_backup_plan` with a cross-region `copy_action`.
- S3 Cross-Region Replication (`aws_s3_bucket_replication_configuration`).
- **State RPO**: the state file itself needs a versioned remote backend (S3 versioning, or Azure Blob versioning plus soft delete) so you can always restore it.
- Note: running backup scripts through provisioners is an anti-pattern. Use managed backup services instead.

RTO - this is where IaC shines:

- Modular, **region-parameterized** code so the same stack can be applied in the DR region just by changing a variable.
- A CI pipeline ready to rebuild the stack in the DR region.
- Pre-provisioned **pilot light / warm standby** resources (replica DB, minimal nodes) already defined in code, to cut rebuild time.
- Monitoring and alerts defined in code so DR comes up already observable.

- **Regular DR drills** where you actually time the rebuild and compare it with the RTO target.

Example:

```
resource "aws_backup_plan" "db" {
  name = "prod-db-daily"
  rule {
    rule_name         = "daily"
    target_vault_name = aws_backup_vault.primary.name
    schedule          = "cron(0 2 * * ? *)"
    lifecycle { delete_after = 35 }
    copy_action {
      destination_vault_arn = aws_backup_vault.dr.arn   # vault in the DR region
      lifecycle { delete_after = 35 }
    }
  }
}

resource "aws_db_instance" "primary" {
  identifier              = "prod-db"
  backup_retention_period = 7          # automated backups + PITR
  # ...
}

resource "aws_db_instance" "dr_replica" {
  provider            = aws.dr          # provider alias for the DR region
  identifier          = "prod-db-dr"
  replicate_source_db = aws_db_instance.primary.arn
  instance_class      = "db.r6g.large"
}
```

> ***What to say to the interviewer:*** *RPO is how much data we can afford to lose and RTO is how quickly we must recover. Terraform does not back up data, but I use it to configure backups, PITR, AWS Backup cross-region copies and replicas for RPO, and I keep state in a versioned backend. For RTO I keep region-parameterized modules, a pipeline that can rebuild in the DR region, pilot-light resources, and I run timed DR drills to prove we meet the target.*

Q33. How do you refactor Terraform code (rename resources, move them into modules) without downtime or recreation?

Also asked as: What happens if you rename a resource in Terraform? How do you use terraform state mv?

> **In simple words:** Terraform tracks every real resource by its "address" in the code, like a house tracked by its street address. If you rename the street without telling the post office, it thinks the old house was demolished and a new one must be built. A `moved` block is the change-of-address form.

Detailed answer

- If you simply rename `aws_instance.web` to `aws_instance.app`, the plan shows **destroy + create**, because the old address disappears from the config and a new one appears.
- `moved` **block (Terraform 1.1+, preferred):** declare the rename in code. It is reviewed in the PR, works for every teammate and pipeline automatically, and can be removed later.
- `terraform state mv` **(imperative):** edits the state directly, once. It's fine for a quick fix, but it is not visible in code review, and every other workspace or environment needs the same command.
- The same applies when you change a `count` resource to `for_each` (index `[0]` -> key `["web"]`), or move resources into a module.

- Always confirm the plan says **"has moved to"** and shows **0 to destroy** before applying. Keep refactor PRs separate from functional changes.

Example:

```
# Renamed a resource and moved another into a module - no recreation
moved {
  from = aws_instance.web
  to   = aws_instance.app
}

moved {
  from = aws_s3_bucket.logs
  to   = module.logging.aws_s3_bucket.this
}

moved {
  from = aws_subnet.private[0]          # count -> for_each migration
  to   = aws_subnet.private["ap-south-1a"]
}
```

```
terraform plan
#  # aws_instance.web has moved to aws_instance.app
#  Plan: 0 to add, 0 to change, 0 to destroy.

# Imperative alternative (one-off, not reviewable):
terraform state mv aws_instance.web aws_instance.app
```

> ***What to say to the interviewer:*** *Renaming a resource or moving it into a module changes its address, so without telling Terraform the plan would destroy and recreate it. I add* `moved` *blocks in the same PR so the refactor is reviewed and applied the same way everywhere, and I check that the plan shows "has moved" and zero destroys.* `terraform state mv` *is my fallback for one-off fixes.*

Q34. How do you perform zero-downtime updates with Terraform?

> **In simple words:** When you replace a bridge, you build the new one next to it, move the traffic across, and only then knock down the old one. By default Terraform knocks down first and builds second.

Detailed answer

- **Know which changes force replacement.** In the plan, `-/+` or "forces replacement" means destroy then create, which is downtime by default. Examples: changing an instance's AMI or subnet, an RDS identifier, some launch template settings.
- `lifecycle { create_before_destroy = true }`: Terraform creates the new resource, repoints dependencies, then deletes the old one. Names must be unique, so use `name_prefix` instead of a fixed `name`.
- **Auto Scaling Groups:** change the launch template version and use `instance_refresh` with `min_healthy_percentage`, so instances roll one batch at a time behind the load balancer health checks.
- **Blue-green with Terraform:** two target groups (or two ASGs) and a weighted ALB listener rule; shift the weights in steps through variables.
- **Databases:** prefer in-place modifications, keep `apply_immediately = false` for the maintenance window, and use RDS Blue/Green Deployments for engine upgrades.

- Protect critical resources with `prevent_destroy`, and review every plan for unexpected replacements.

Example:

```
resource "aws_launch_template" "app" {
  name_prefix   = "app-"
  image_id      = var.ami_id
  instance_type = "t3.medium"
  lifecycle { create_before_destroy = true }
}

resource "aws_autoscaling_group" "app" {
  name_prefix         = "app-"
  min_size            = 2
  max_size            = 6
  vpc_zone_identifier = var.private_subnet_ids
  target_group_arns   = [aws_lb_target_group.app.arn]
  health_check_type   = "ELB"

  launch_template {
    id      = aws_launch_template.app.id
    version = aws_launch_template.app.latest_version
  }

  instance_refresh {
    strategy = "Rolling"
    preferences { min_healthy_percentage = 90 }
  }
}
```

What to say to the interviewer: *First I read the plan for anything marked "forces replacement". For those I use* `create_before_destroy` *with* `name_prefix`*, so the new resource exists before the old one is removed. For compute behind a load balancer I roll through an ASG instance refresh that keeps 90% healthy, and for risky changes I do blue-green with weighted target groups. Databases get in-place changes or RDS Blue/Green.*

Q35. What are circular dependencies in Terraform, and how do you fix them?

In simple words: Two people each saying "I'll sign the contract after you sign it". Nobody can go first, so nothing happens. Terraform stops with "Error: Cycle".

Detailed answer

- Terraform builds a dependency graph from references (and `depends_on`). If A needs B and B needs A, there is no valid order, and plan fails with `Error: Cycle: aws_security_group.a, aws_security_group.b`.
- **Classic case:** two security groups with inline rules that reference each other (app allows DB, DB allows app).
- **Fix: split the relationship into separate resources.** Create both groups with no rules, then add the rules as standalone `aws_vpc_security_group_ingress_rule` resources. The groups no longer depend on each other; the rules depend on both.
- Other fixes: move a value into a variable or data source instead of a resource reference, remove an unnecessary `depends_on`, or split modules whose outputs feed each other.
- Use `terraform graph | dot -Tsvg > graph.svg` to see the loop.

Example:

```
# BAD: inline rules referencing each other -> Error: Cycle
# resource "aws_security_group" "app" { ingress { security_groups = [aws_security_group.db.id] } }
# resource "aws_security_group" "db"  { ingress { security_groups = [aws_security_group.app.id] } }

# GOOD: groups first, rules as separate resources
resource "aws_security_group" "app" {
  name_prefix = "app-"
  vpc_id      = var.vpc_id
}
resource "aws_security_group" "db" {
  name_prefix = "db-"
  vpc_id      = var.vpc_id
}

resource "aws_vpc_security_group_ingress_rule" "db_from_app" {
  security_group_id            = aws_security_group.db.id
  referenced_security_group_id = aws_security_group.app.id
  from_port                    = 5432
  to_port                      = 5432
  ip_protocol                  = "tcp"
}
```

What to say to the interviewer: *A cycle happens when two resources reference each other, so Terraform can't decide what to create first and fails with "Error: Cycle". The typical case is two security groups with inline rules pointing at each other. I fix it by creating the groups without rules and adding the rules as separate resources, which breaks the loop.* `terraform graph` *helps me see where the loop is.*

Scenario-Based

Q36. A resource was manually changed in AWS (for example someone added an S3 bucket policy to a Terraform-managed bucket), and Terraform shows drift. How do you handle it using IaC?

Also asked as: If someone manually changed an S3 bucket policy that was originally created with Terraform, how would you deal with that drift?

Also asked as: Someone changed a resource manually in the console. How do you fix the drift?

In simple words: Either put the cake back the way the recipe says, or, if the change was good, update the recipe. Never leave them different.

Detailed answer

Steps:

1. **See the drift**: `terraform plan -refresh-only` shows the policy change.
2. **Find the reason**: CloudTrail `PutBucketPolicy` event - who, when, why (maybe an urgent access fix).
3. **Decide**:

- Unwanted / insecure (e.g. made bucket public) -> run `terraform apply` to overwrite it with the code version. If the policy resource was never in code, add an `aws_s3_bucket_policy` resource with the correct policy so Terraform owns it.

- Intended change -> copy the policy into code (`aws_s3_bucket_policy` + `aws_iam_policy_document`), raise a PR, and plan until "No changes".
- If the policy object exists but is not in state -> `import` it.

4. **Prevent**: prod console is read-only, SCP or IAM deny on `s3:PutBucketPolicy` except for the Terraform role, scheduled drift detection, AWS Config rule for public buckets.

Note: in AWS provider v4+, bucket policy is a separate resource `aws_s3_bucket_policy`; do not also define it inline elsewhere or two resources will fight.

Example:

```
data "aws_iam_policy_document" "assets" {
  statement {
    actions   = ["s3:GetObject"]
    resources = ["${aws_s3_bucket.assets.arn}/*"]
    principals {
      type        = "AWS"
      identifiers = ["arn:aws:iam::222222222222:role/app"]
    }
  }
}

resource "aws_s3_bucket_policy" "assets" {
  bucket = aws_s3_bucket.assets.id
  policy = data.aws_iam_policy_document.assets.json
}
```

```
terraform import aws_s3_bucket_policy.assets acme-prod-assets   # if not yet managed
terraform plan   # goal: No changes
```

> ***What to say to the interviewer:*** *I first run plan -refresh-only and check CloudTrail to see who changed the policy and why. If the change was wrong I re-apply the code to revert it; if it was needed I codify it in an aws_s3_bucket_policy resource, import it if needed, and get the plan to show no changes. Then I lock down console write access and schedule drift detection so it does not happen again.*

Q37. There is an existing VPC created manually. How would you import it into Terraform without breaking anything? What are the prerequisites and how do you pass the VPC ID?

Also asked as: What are the prerequisites before importing a VPC? How do you pass arguments to a VPC with terraform import?

> **In simple words:** You are adopting a house that is already built. You measure it, draw the blueprint exactly as it is, and register it - you do not knock anything down.

Detailed answer

Prerequisites:

- Terraform and AWS provider configured with credentials that can read the VPC (and backend set up for state).
- The real IDs: VPC ID (`vpc-0abc...`), plus IDs of subnets, route tables, IGW, NAT, SGs you also want to manage.

- Know the current settings (CIDR, DNS support, tags) - get them from `aws ec2 describe-vpcs`.
- Take a snapshot of current config and decide scope (import only the VPC, or all its children).

How the ID is passed: `terraform import` takes two arguments - the resource address in code and the real ID. You do not pass CIDR etc. on the command line; those come from your code, which must match reality.

Safe steps:

1. Write an `import` block (or empty resource for the CLI method).
2. `terraform plan -generate-config-out=vpc.tf` to generate matching code.
3. Tidy the code; keep values identical to the real VPC.
4. `terraform plan` must show `1 to import, 0 to add, 0 to change, 0 to destroy`. Any "must be replaced" (for example a CIDR mismatch) means your code is wrong - fix it, never apply.
5. Apply, then import subnets, route tables, IGW the same way.
6. Add `lifecycle { prevent_destroy = true }` for safety.

Example:

```
import {
  to = aws_vpc.main
  id = "vpc-0abc123def4567890"
}

resource "aws_vpc" "main" {
  cidr_block           = "10.20.0.0/16"
  enable_dns_support   = true
  enable_dns_hostnames = true
  tags                 = { Name = "legacy-prod-vpc" }

  lifecycle {
    prevent_destroy = true
  }
}
```

```
# CLI alternative: address + real ID are the two arguments
terraform import aws_vpc.main vpc-0abc123def4567890
terraform plan    # must be: 0 to add, 0 to change, 0 to destroy
```

> ***What to say to the interviewer:*** *I gather the real VPC ID and its settings, write an import block or run terraform import with the resource address and the VPC ID, and generate or write matching code. I only apply once the plan shows zero changes and zero destroys, then repeat for subnets and route tables, and protect it with prevent_destroy.*

Q38. Your Terraform state file was accidentally deleted, lost, or got corrupted / out of sync. How do you recover it safely? What are the risks and how do you protect it?

Also asked as: A team member accidentally deleted the state file. What immediate steps prevent drift or resource loss?

Also asked as: Scenario: the state file got deleted. What do you do?

In simple words: If the notebook is lost, Terraform thinks nothing exists and will try to build everything again. You either restore the notebook from a backup copy or rewrite it by re-adopting each resource.

Detailed answer

Immediate: **freeze all applies first** - lock or pause the pipeline (and keep/obtain the state lock) so nobody runs an apply against an empty state and creates duplicates or orphans. Then restore the **previous S3 object version** of the state before doing anything else.

Recovery options:

1. **Restore from backend versioning** (best): S3 versioning -> restore the previous object version; GCS object versioning; HCP Terraform keeps state history.
2. **Local backup**: `terraform.tfstate.backup` or a copy from `terraform state pull` done earlier.
3. **Corrupted but present**: `terraform state pull > broken.json`, fix JSON carefully (or restore an older version), bump the serial, `terraform state push`.
4. **No backup at all**: rebuild state by importing every resource (import blocks, or tools like terraformer) until `plan` shows no changes.
5. **Out of sync**: `terraform plan -refresh-only`, then `state rm` for things that no longer exist and `import` for things missing from state.

Risks: duplicate resources, apply trying to recreate or delete prod, orphaned resources nobody tracks, secrets exposed if state leaks.

Protection:

- Remote backend with **versioning**, KMS encryption, MFA delete / object lock, bucket deletion protection.
- Locking to prevent corruption from parallel runs.
- Least-privilege IAM on the state bucket; only CI can write.
- Separate state per env/component (smaller blast radius).
- Never edit state by hand; use `state mv/rm/import`.

Example:

```
aws s3api list-object-versions --bucket acme-tf-state --prefix prod/network/terraform.tfstate
aws s3api get-object --bucket acme-tf-state --key prod/network/terraform.tfstate \
  --version-id 3HL4kqtJlcpXroDTDmJ.rmSpXd3dIbrHY restored.tfstate
terraform state push restored.tfstate
terraform plan    # verify: No changes
```

What to say to the interviewer: *First I freeze all pipelines, then restore the last good version from S3 versioning and push it, and verify with a plan that shows no changes. If there is no backup I rebuild the state by importing each resource. To protect it, the backend has versioning, encryption, locking, restricted access, and state is split per environment so any damage stays small.*

Q39. A terraform apply fails midway in production after creating some resources. What would be your next steps?

Also asked as: Scenario: terraform apply failed halfway. How do you recover?

In simple words: The builder stopped halfway. Do not panic and restart from scratch; check what got built, fix the problem, and let Terraform finish the rest.

Detailed answer

Terraform is not transactional - there is no automatic rollback. Whatever was created before the error is already recorded in state.

Steps:

1. **Do not re-run blindly.** Read the error (quota limit, IAM permission, name conflict, timeout, API throttling, dependency).
2. **Check the lock**: if the process crashed, a stale lock may remain -> confirm no run is active, then `force-unlock`.
3. **Check state vs reality**: `terraform plan` shows what is done and what is still pending. Look for resources marked tainted (created but failed during setup) - Terraform will replace them.
4. **Check for orphans**: if a resource was created in the cloud but the crash stopped state from being saved, it is not in state -> import it (otherwise the next apply fails with "already exists" or creates a duplicate).
5. **Fix the root cause**: raise quota, fix IAM, fix the config.
6. **Re-run plan, review, apply** - Terraform continues from where it stopped.
7. If the partial change is hurting production, roll back by reverting the code commit and applying the previous version (or restoring state + infra).
8. Afterwards: add smaller changes, pre-checks (quotas, permissions), and test in stage first.

Example:

```
Error: creating EC2 Instance: VcpuLimitExceeded: You have requested more vCPU capacity...
```

```
terraform plan          # 2 already created, 3 still to add
# fix quota / config, then
terraform apply
```

What to say to the interviewer: *Terraform does not roll back, so I first read the error, clear a stale lock if needed, and run plan to see exactly what was created and what is pending, importing any orphaned resources that were created but not recorded. After fixing the root cause I re-plan and apply to finish, or revert the commit and apply if the partial state is hurting production.*

Q40. terraform plan shows that critical resources will be deleted unexpectedly. What would you do?

Also asked as: During apply, production resources are about to be recreated. How do you detect and stop it before impact?

In simple words: The builder says "I will tear down your main pillar". Stop, do not approve, and find out why he thinks that.

Detailed answer

1. **Do not apply.** Stop the pipeline.
2. Read the plan: is it `destroy` or `must be replaced` (`-/+`)? The plan tells which attribute "forces replacement".

3. Common causes and fixes:

Cause	Fix
Resource renamed / moved into a module	`moved` block or `terraform state mv`
count index shifted (item removed from a list)	switch to `for_each` with stable keys + `moved`
Immutable field changed (DB identifier, AZ, CIDR, name)	revert the change or plan a proper migration
Wrong workspace / wrong backend / wrong tfvars	check `terraform workspace show` and backend key
Provider major upgrade changed behavior	pin version, read upgrade guide
Resource removed from code but still needed	restore the code; to just stop managing, use a `removed` block or `state rm`
Drift - someone changed it manually	codify or revert

4. Re-plan until destroy count is 0 for critical resources.
5. Prevent: `lifecycle { prevent_destroy = true }`, `deletion_protection = true` on RDS/ALB, policy checks in CI that fail on destroys, mandatory reviewer for prod.

Detecting and stopping it before impact:

- Read the plan symbols: `-/+` means destroy then create, `+/-` means create before destroy, and the attribute line says `# forces replacement`. The summary line shows `N to destroy`.
- Always `terraform plan -out=tfplan`, review it, and apply only that saved plan - nothing new can sneak in between review and apply.
- CI policy gate: `terraform show -json tfplan` + conftest/OPA fails the pipeline if any action contains `delete` for protected types (RDS, S3, EKS).
- lifecycle options: `prevent_destroy` makes the plan error out; `create_before_destroy` avoids downtime when replacement is truly needed; `ignore_changes` stops replacement for attributes changed by other systems.
- If you notice it mid-apply, press **Ctrl+C once**: Terraform stops starting new operations, lets in-flight ones finish, saves state and releases the lock. Pressing it a second time forces exit and can leave state inconsistent and the lock stuck, so avoid that.

Example:

```
moved {
  from = aws_db_instance.db
  to   = module.database.aws_db_instance.this
}

resource "aws_db_instance" "this" {
  # ...
  deletion_protection = true
  lifecycle {
    prevent_destroy = true
  }
}
```

> ***What to say to the interviewer:*** *I never apply; I read the plan to find which attribute forces replacement or why the resource is being destroyed, which is usually a rename, a count index shift, an immutable field change or the wrong workspace. I fix it with moved blocks or state mv until the plan shows no destroy, and I protect critical resources with prevent_destroy, deletion_protection and*

a CI policy that blocks destroys.

Q41. A Terraform script bypassed validation and deleted a production database (or other prod resources) during peak business hours. What is your action plan, and how do you contain the blast radius?

Also asked as: How would you troubleshoot and contain the blast radius if Terraform accidentally deleted production resources?

In simple words: First stop the fire from spreading, then bring the patient back from the latest backup, keep everyone informed, and afterwards fix the process so nobody can do it again.

Detailed answer

Phase 1 - Contain (first minutes):

1. Declare an incident, open a war room, assign an incident commander and a communicator (status page / stakeholders).
2. **Stop further damage**: cancel running pipelines, disable the Terraform CI job, revoke/limit the Terraform role, keep the state lock.
3. Find scope: what else was in the plan? Check apply logs and CloudTrail (`DeleteDBInstance`, who/when).

Phase 2 - Recover:

4. Restore the database:

- RDS: restore from the **final snapshot** (if `skip_final_snapshot = false`), latest automated snapshot, or **point-in-time restore** to just before deletion. Retained automated backups can exist after deletion if configured.
- Aurora: restore cluster from snapshot / backtrack if enabled.
- Or promote a cross-region read replica / AWS Backup copy.

5. Point the app to the restored endpoint (same identifier, or update DNS/secret), put the app in maintenance mode meanwhile.
6. Re-align Terraform: import the restored DB into state so the next plan is clean.
7. Validate data (row counts, last transactions), then reopen traffic and monitor.

Phase 3 - Prevent (post-mortem, blameless):

- `deletion_protection = true` and `prevent_destroy` on all stateful resources; `skip_final_snapshot = false`.
- Split state: databases in their own state, rarely touched.
- Policy as code (OPA/Sentinel) that fails any plan with destroy on data stores.
- Mandatory plan review + approval; apply only saved plans; no applies from laptops; freeze during peak hours.
- IAM guardrails: SCP or deny `rds:DeleteDBInstance` for the pipeline role except via break-glass.
- Tested backups: AWS Backup, cross-region/cross-account copies, regular restore drills.

Example:

```
aws rds restore-db-instance-to-point-in-time \
  --source-db-instance-identifier prod-db \
  --target-db-instance-identifier prod-db-restored \
  --restore-time 2026-09-20T10:14:00Z
terraform import aws_db_instance.prod prod-db-restored
```

```
{
  "Effect": "Deny",
  "Action": ["rds:DeleteDBInstance", "rds:DeleteDBCluster"],
  "Resource": "*",
  "Condition": { "StringNotLike": { "aws:PrincipalArn": "arn:aws:iam::*:role/BreakGlass" } }
}
```

> ***What to say to the interviewer:*** *I would first contain it by declaring an incident, stopping all pipelines and restricting the Terraform role, then restore the database from the final snapshot or point-in-time recovery, reconnect the app, and import the restored DB back into state. In the post-mortem I would add deletion_protection and prevent_destroy, move databases to their own state, block destroys with OPA policies and an IAM deny, and require approved saved plans.*

Q42. Terraform creates an IAM role and immediately uses it, but the apply fails even though the trust policy and permissions are correct. Why, and how do you fix it?

> **In simple words:** IAM is a global notice board. When you pin a new notice, it takes a few seconds before every office in the world can read it. If the next step runs too fast, it cannot see the notice yet.

Detailed answer

Cause: **IAM eventual consistency**. IAM changes are replicated globally; a new role, trust policy or policy attachment can take a few seconds to become visible to other services (Lambda, EKS, EC2 instance profiles, ECS).

Typical error:

```
Error: creating Lambda Function (app): InvalidParameterValueException:
The role defined for the function cannot be assumed by Lambda.
```

Other variants: `InvalidInstanceProfile` / "Invalid IAM Instance Profile name" on EC2, access denied right after attaching a policy.

Fixes:

1. **Re-run first**: the second apply usually succeeds - confirms it is propagation, not a config bug.
2. The AWS provider already has **built-in retries** for many of these cases; keep the provider up to date.
3. Add a **time_sleep** after the role/policy attachment and make the consumer depend on it.
4. Use **depends_on** so the consumer waits for the policy attachment, not just the role (a common real bug: the Lambda referenced the role, but not the attachment).
5. In scripts/pipelines, **retry with backoff** instead of failing immediately.
6. Longer term: create shared IAM roles in an earlier stack so they already exist when apps are deployed.

Example:

```
resource "aws_iam_role_policy_attachment" "lambda_basic" {
  role       = aws_iam_role.lambda.name
  policy_arn = "arn:aws:iam::aws:policy/service-role/AWSLambdaBasicExecutionRole"
}

resource "time_sleep" "iam_propagation" {
  depends_on      = [aws_iam_role_policy_attachment.lambda_basic]
  create_duration = "15s"
}

resource "aws_lambda_function" "app" {
  depends_on    = [time_sleep.iam_propagation]
  function_name = "app"
  role          = aws_iam_role.lambda.arn
  runtime       = "python3.12"
  handler       = "app.handler"
  filename      = "app.zip"
}
```

> ***What to say to the interviewer:*** *That is IAM eventual consistency: the role exists but has not propagated to the service yet, which gives errors like "The role defined for the function cannot be assumed by Lambda". I confirm it by re-running, then fix it by making the consumer depend on the policy attachment and adding a short time_sleep, relying on the provider's built-in retries, and in the long run creating shared IAM roles in an earlier stack.*

Q43. Scenario: you need to create dynamic IAM roles across multiple environments. What is your module strategy?

> **In simple words:** Build one "role factory" module. Each environment hands it a shopping list of roles, and the factory stamps out correctly named, safely limited roles for that environment.

Detailed answer

Strategy:

1. **One reusable** `iam-role` **module** that takes a **map of roles** as input (name -> trusted principals, managed policies, inline policy statements).
2. Inside, `for_each` over the map, so adding or removing a role never shifts others.
3. **Env-prefixed names** (`dev-app-reader`, `prod-app-reader`) built from a `var.env` in locals, plus common tags.
4. A **permissions boundary** on every role, so even a bad policy cannot exceed the allowed maximum.
5. Policies built with **aws_iam_policy_document** (validated HCL, no hand-written JSON strings).
6. **Per-environment tfvars** (`dev.tfvars`, `prod.tfvars`) holding the role map for that env.
7. **One module call per account** using a provider alias with assume_role (dev account, prod account).
8. Guardrails: CI policy check (no `*:*`, no wildcard principals), pinned module version, review on changes.

Example:

```
# modules/iam-role/main.tf
variable "env" { type = string }
variable "permissions_boundary_arn" { type = string }
variable "roles" {
  type = map(object({
    trusted_services = list(string)
    policy_arns      = list(string)
  }))
}

data "aws_iam_policy_document" "trust" {
  for_each = var.roles
  statement {
    actions = ["sts:AssumeRole"]
    principals {
      type        = "Service"
      identifiers = each.value.trusted_services
    }
  }
}

resource "aws_iam_role" "this" {
  for_each             = var.roles
  name                 = "${var.env}-${each.key}"
  assume_role_policy   = data.aws_iam_policy_document.trust[each.key].json
  permissions_boundary = var.permissions_boundary_arn
  tags                 = { Env = var.env, ManagedBy = "terraform" }
}

locals {
  attachments = merge([
    for role, cfg in var.roles : {
      for arn in cfg.policy_arns : "${role}|${arn}" => { role = role, arn = arn }
    }
  ]...)
}

resource "aws_iam_role_policy_attachment" "this" {
  for_each   = local.attachments
  role       = aws_iam_role.this[each.value.role].name
  policy_arn = each.value.arn
}

output "role_arns" {
  value = { for k, r in aws_iam_role.this : k => r.arn }
}
```

```
# envs/prod/main.tf
module "iam_roles" {
  source                   = "git::https://github.com/acme/tf-modules.git//iam-role?ref=v2.1.0"
  providers                = { aws = aws.prod }
  env                      = "prod"
  permissions_boundary_arn = "arn:aws:iam::333333333333:policy/DevOpsBoundary"
  roles                    = var.roles   # from prod.tfvars
}
```

```
# prod.tfvars
roles = {
  app-lambda = {
    trusted_services = ["lambda.amazonaws.com"]
    policy_arns      = ["arn:aws:iam::aws:policy/service-role/AWSLambdaBasicExecutionRole"]
  }
  batch-ec2 = {
    trusted_services = ["ec2.amazonaws.com"]
    policy_arns      = ["arn:aws:iam::aws:policy/AmazonSSMManagedInstanceCore"]
  }
}
```

> ***What to say to the interviewer:*** *I build one reusable iam-role module that takes a map of roles and uses for_each, with env-prefixed names, a mandatory permissions boundary and policies written with aws_iam_policy_document. Each environment has its own tfvars with the role map, and I call the module once per account through a provider alias, with the module version pinned and CI policy checks blocking wildcard permissions.*

Q44. Scenario: AWS API rate limits (throttling) are hit during terraform apply. How do you handle it?

> **In simple words:** Terraform is sending AWS too many requests at the same time, like 50 people shouting at one cashier. You slow the queue down, let the cashier retry calmly, and split the big order into smaller ones.

Detailed answer

How you recognize it: errors like `ThrottlingException: Rate exceeded` or `RequestLimitExceeded`, very slow applies, or the same apply succeeding on a retry. Run with `TF_LOG=DEBUG` to see the retries and which API is being throttled.

Fixes, in the order I would try:

1. **Lower parallelism**: `terraform apply -parallelism=N` (default is 10) so fewer API calls run at once.
2. **Tune provider retries**: in the `aws` provider block set `max_retries` and `retry_mode = "adaptive"`, which backs off automatically on throttling.
3. **Split into smaller root modules/stacks**: fewer resources per plan means fewer describe/refresh calls (big states also refresh hundreds of resources on every plan).
4. **Avoid huge fan-out at once**: a `for_each` creating 500 records or roles in one run hammers the same API; roll it out in batches.
5. **Request a quota increase** (Service Quotas) if a real limit is hit, and check whether other tools in the same account (CI jobs, scripts, autoscalers) share the same API budget.
6. `depends_on` / `time_sleep` to serialize calls are a crude last resort, not the main fix.

Example:

```
provider "aws" {
  region      = "ap-south-1"
  max_retries = 25
  retry_mode  = "adaptive"   # client-side rate limiting with backoff
}
```

```
TF_LOG=DEBUG terraform apply -parallelism=3 2> tf-debug.log
grep -i "throttl\|rate exceeded" tf-debug.log | head
```

> ***What to say to the interviewer:*** *I confirm it is throttling from errors like ThrottlingException or Rate exceeded in TF_LOG=DEBUG output, then reduce -parallelism below the default of 10 and set max_retries with retry_mode adaptive in the AWS provider. Longer term I split large states into smaller stacks, avoid creating hundreds of resources in one run, and request quota increases where needed; time_sleep or depends_on is only a crude last resort.*

Q45. Scenario: you added a new EC2 instance to an Auto Scaling Group using Terraform, but the ASG does not register the instance. How do you troubleshoot?

> **In simple words:** An Auto Scaling Group is a factory that builds its own cars from its own blueprint (the launch template). A car you built separately in your garage does not become part of the factory fleet just because it looks the same.

Detailed answer

Key point: an `aws_instance` resource **never joins an ASG automatically**. ASGs launch and own their instances from the launch template. So the right fix depends on what you wanted:

- **Want more instances** -> change `desired_capacity` / `min_size` on the ASG (or let a scaling policy do it).
- **Want new settings / new AMI** -> change the launch template and bump its version, then roll out with an **instance refresh**.
- **Really want to attach an existing instance** -> `aws autoscaling attach-instances` (CLI/API). Terraform has no first-class resource for attaching EC2 instances; `aws_autoscaling_attachment` is for attaching the ASG to ELBs / target groups, not instances.

If the ASG itself is not launching or keeping instances, check:

1. **Scaling activity history**: `aws autoscaling describe-scaling-activities` - it states exactly why a launch failed.
2. **max_size reached** - desired cannot go above max.
3. **Subnets / AZs**: `vpc_zone_identifier` subnets must be valid, in the right VPC, and have free IPs; an attached instance must be in one of the ASG's AZs.
4. **Launch template valid**: AMI exists in this region, instance profile exists, security groups belong to the same VPC, key pair exists.
5. **Health checks**: with `health_check_type = "ELB"` and a short grace period, new instances are marked unhealthy and terminated before the app starts.
6. **Lifecycle hooks** stuck in `Pending:Wait` until timeout.
7. **Terraform masking changes**: `lifecycle { ignore_changes = [desired_capacity] }` means your capacity change in code is silently ignored.
8. **Instance refresh** in progress or failed, or suspended processes (`Launch` suspended).
9. **Quotas / capacity**: vCPU service quota reached or `InsufficientInstanceCapacity` for that instance type in the AZ -> use multiple instance types or AZs.

Example:

```
resource "aws_autoscaling_group" "web" {
  name                = "web-asg"
  min_size            = 2
  max_size            = 6
  desired_capacity    = 3              # change this to add instances
  vpc_zone_identifier = var.private_subnets
  health_check_type         = "ELB"
  health_check_grace_period = 300

  launch_template {
    id      = aws_launch_template.web.id
    version = aws_launch_template.web.latest_version
  }

  instance_refresh {
    strategy = "Rolling"
    preferences { min_healthy_percentage = 90 }
  }
}
```

```
aws autoscaling describe-scaling-activities --auto-scaling-group-name web-asg --max-items 5
aws autoscaling attach-instances --instance-ids i-0abc123 --auto-scaling-group-name web-asg
```

> ***What to say to the interviewer:*** *An aws_instance created by Terraform never joins an ASG on its own, because the ASG launches its own instances from the launch template, so I would raise desired_capacity or update the launch template version, or use attach-instances if I really need to adopt an existing instance. If the ASG is not launching, I check the scaling activity history, max_size, subnets and AZs, launch template validity, health check grace period, lifecycle hooks, ignore_changes on desired_capacity, and capacity or quota errors.*

Q46. What is the fastest way to roll back an infrastructure change made with Terraform?

> **In simple words:** Terraform has no undo button. Your Git history is the undo button - go back to the last good recipe and cook again.

Detailed answer

Key facts:

- There is **no** `terraform rollback` **command**. Terraform only knows "make reality match the code". So rolling back = putting the old code back and applying it.
- **Fastest safe path**: `git revert` the bad commit -> pipeline runs `plan` -> review that the plan only undoes the bad change -> `apply`.
- Because modules are versioned, rolling back a module change is often just pinning `?ref=` back to the previous tag.
- `-target` lets you apply only one resource. Use it only in an emergency to fix the broken piece quickly, then run a full plan/apply afterwards so nothing is left out of sync. It is not for routine use.
- Some changes cannot be "rolled back" by code alone: a deleted database or a replaced resource with new IDs needs a restore from snapshot/backup.
- **Restoring an older state version from S3** is the last resort, only when state itself is broken - it does not change real infrastructure and can make state lie about reality.

Real example: a security group update removed the ingress rule the ALB needed, and the service started failing health checks. I reverted the Git commit, the pipeline showed a plan adding back just that one rule, I applied it, and the service recovered within minutes. The post-mortem added a policy check for removing ingress on prod SGs.

Example:

```
git revert a1b2c3d            # the faulty SG change
git push                      # pipeline runs plan
terraform plan -out=tfplan    # expect: 0 to add, 1 to change, 0 to destroy
terraform apply tfplan

# Emergency only, then do a full plan/apply afterwards:
terraform apply -target=aws_security_group.web
```

> ***What to say to the interviewer:*** *Terraform has no rollback command, so the fastest safe rollback is reverting the Git commit and running plan and apply through the pipeline, which restores the previous desired state. I use -target only in emergencies, restoring an old state version from S3 is a last resort for broken state, and for destructive changes like a deleted database I restore from backups. For example, when a security group change broke service access, reverting the commit and re-applying fixed it in minutes.*

Q47. Why might terraform plan show NO changes even though someone modified the infrastructure outside Terraform?

> **In simple words:** Terraform only compares the things written in its checklist. If someone repainted a wall that isn't on the checklist, the inspector happily says "everything matches".

Detailed answer

A common model answer says "because you didn't run terraform refresh". That is **wrong**: `terraform plan` refreshes state from the real cloud by default, so external changes to attributes Terraform manages **do** show up. When plan says "No changes", the real reasons are:

1. **The attribute isn't in your config.** Optional or computed attributes you never set are not enforced. If someone changes them, Terraform doesn't care.
2. `lifecycle { ignore_changes = [...] }` covers that attribute (common for `desired_capacity`, `tags`, `image`).
3. **The change is on a different object Terraform doesn't manage.** For example, someone added an extra security group rule outside Terraform, while your code manages the SG with separate rule resources. Or a bucket policy was added while your code never declared an `aws_s3_bucket_policy`.
4. **Plan ran with** `-refresh=false` (some CI setups do this for speed).
5. **Wrong target:** a different workspace, state file, AWS account/profile or region than the one that was modified.
6. **Provider behaviour:** some attributes aren't read back by the provider, or are normalized so the diff is hidden.

Example:

```
terraform workspace show                  # right workspace?
aws sts get-caller-identity               # right account?
terraform plan -refresh-only              # shows what changed in reality vs state
grep -rn "ignore_changes" .               # is the attribute ignored?
terraform state show aws_security_group.app   # what Terraform actually tracks
```

> ***What to say to the interviewer:*** *Plan refreshes by default, so "no changes" usually means Terraform doesn't manage what was changed: the attribute isn't set in the config, it's in* `ignore_changes`*, or the change was on a separate object like an extra rule or policy. I also check that I'm on the right workspace, account and region, and that CI isn't using* `-refresh=false`*.* `terraform plan -refresh-only` *shows exactly what drifted.*

Q48. Terraform state is huge (200MB) and plan takes 12 minutes. How do you fix it?

> **In simple words:** One giant notebook for the whole city means every small question requires reading every page. Split it into one notebook per neighbourhood, and each question gets answered in seconds.

Detailed answer

The root cause is almost always **one state managing too much**: every plan refreshes thousands of resources through cloud APIs.

1. **Split the state by lifecycle and blast radius:** network (VPC, TGW), data (RDS, S3), platform (EKS), then one stack per service or team. Things that change daily shouldn't live with things that change yearly.
2. **Connect stacks with outputs, not one big graph:** `terraform_remote_state`, or better, publish IDs to SSM Parameter Store and read them with data sources.
3. **Move resources without recreating them:** `removed { lifecycle { destroy = false } }` in the old stack plus `import` blocks in the new one (or `terraform state mv -state-out` for local migrations). Plan both sides to confirm there are 0 destroys.
4. **Don't manage high-churn, high-count objects in Terraform:** thousands of DNS records, S3 objects or IAM users generated by apps belong in app-level tooling.
5. **Quick wins while you refactor:** `-parallelism=20` (watch API throttling), a PR-time plan with `-refresh=false` and a scheduled full refresh plan for drift, avoid large `data` sources that list everything, and keep the backend in the same region as CI.
6. **Tooling for many stacks:** Terragrunt, Terramate, Spacelift or Terraform Cloud workspaces with run triggers.
7. Use `-target` only for emergencies, never as the normal workflow.

Example:

```
infra/
  00-network/     state: network.tfstate   (VPC, subnets, TGW)   -> changes yearly
  10-data/        state: data.tfstate      (RDS, S3, KMS)        -> changes monthly
  20-eks/         state: eks.tfstate       (cluster, node pools)
  30-services/
     billing/     state: billing.tfstate   -> changes daily, plan in ~20s
     meter-api/   state: meter-api.tfstate
```

```
# In 20-eks: read network IDs published by 00-network
data "aws_ssm_parameter" "private_subnets" {
  name = "/infra/network/private_subnet_ids"
}
locals {
  private_subnet_ids = split(",", data.aws_ssm_parameter.private_subnets.value)
}
```

What to say to the interviewer: *A 200MB state means one stack is doing too much, so every plan refreshes thousands of resources. I split it into stacks by lifecycle and blast radius (network, data, cluster, per-service), connect them through SSM parameters or remote state, and move resources with removed and import blocks so nothing is recreated. Meanwhile I tune parallelism and avoid heavy data sources. After the split, most plans run in under a minute and a mistake can only affect one small stack.*